

队伍编号	MC25006794
题号	B

基于混合整数线性规划和改进分支定界法的平移置换搬迁决策模型

摘 要

随着我国城镇化建设全面进入提质增效的存量更新阶段,城市更新对城市面貌和居住品质的提升具有重要意义。目前,平移置换策略的提出在很大程度上提高了房屋腾退率和城市更新率。然而,这一策略在老城街区搬迁决策的实施过程中,仍受到住户搬迁补偿和开发商租金收益的限制。针对这一问题,本文提出了一种基于**混合整数线性规划模型(MILP)**和改进分支定界法的智能决策方法。

针对问题一,考虑面积补偿、采光补偿、修缮补偿、房屋密集程度、距离街道距离和**搬迁距离**等评价指标,通过对指标进行定量化和归一化,其中,房屋密集程度采用**空间句法定量化**。然后,在面积补偿、采光补偿、修缮补偿的约束下,建立了**搬迁补偿综合评价模型**,并设计了合理的搬迁补偿方案。根据满意度阈值,模型可以为每个待搬地块筛选出符合满意度标准的空置地块。最后,针对搬迁补偿的其他影响因素,从规划局/开发商的角度探讨了应对策略。

针对问题二,考虑整院收益、**毗邻加成**收益、搬迁成本等经济指标,通过设置搬迁决策、整院判定、毗邻关系及修缮补偿四类**决策变量**,以最大化开发商净收益为目标建立了**MILP**模型,从面积补偿、采光补偿、修缮补偿、地块容量等角度设定了12种**约束条件**,以确保方案的可行性。然后,设计了结合Yalmip+Gurobi求解工具并利用**分支定界法**求解模型得到了最优搬迁方案与总收益。在分支定界求解过程中,结合**约束传播与割平面法**缩小解空间,提高了计算效率。最后,利用可视化工具进行成本**敏感性分析**等数据分析,验证了出租时间对面积补偿成本的显著影响。

针对问题三,在问题二模型的基础上建立了考虑性价比的**MILP**模型,进一步具体化了目标函数的表达场景。根据所给信息建立了**搬迁规模与性价比 m 的动态关联机制**。为定性分析性价比 m ,引入腾空院落数-性价比曲线分析,并据此设计了逐步模拟搬迁进程的**迭代算法**。通过计算每一步新增成本与收益的**边际效应**,绘制 m 值随腾空院落数变化的趋势曲线并找到性价比的**拐点**,输出了该条件下的搬迁方案与总收益。结果显示,相比于以单纯的总效益最大为目标得到的搬迁方案,成本的减少尤其是面积损失的减少能带来更大的性价比,印证了**拐点**后收益增长速率低于成本攀升趋势。

针对问题四,针对问题四,设计了基于参数驱动的智能老城区平移置换决策软件框架。软件主要采用**Figma**前端用户界面与**Matlab-Gurobi**后台决策程序协同架构,具有地块数据预处理、用户角色匹配、多目标优化决策、定量指标计算及结果可视化等核心功能。前端输入**参数体系**涵盖基础数据、住户偏好参数、开发商策略参数,后台决策程序通过角色匹配自动调用问题一(搬迁补偿模型)、问题二(保底成本最优模型)或问题三(投资拐点分析模型)求解,实现了**搬迁路径智能规划**。

关键词: 平移置换; 智能决策; MILP 模型; 分支定界法; 搬迁补偿; Gurobi;

目录

一、问题重述.....	1
1.1 问题背景.....	1
1.2 问题提出.....	1
二、模型假设与符号说明	2
2.1 模型假设.....	2
2.2 符号说明.....	2
2.3 数据说明.....	4
三、问题一的模型建立与求解.....	5
3.1 问题分析.....	5
3.2 建立居民搬迁意愿评价模型	5
3.3 补偿应对策略	8
3.4 模型求解.....	9
四、问题二的模型建立与求解.....	9
4.1 问题分析.....	9
4.2 建立整院数量最大化与毗邻效益协同优化模型.....	10
4.2.1 决策变量	10
4.2.2 目标函数	11
4.2.3 约束条件	11
4.3 模型求解.....	13
4.4 结果分析.....	14
五、问题三的模型建立与求解.....	19
5.1 问题分析.....	19
5.2 建立性价增益的搬迁性价比拐点规划模型	20
5.3 模型求解.....	22
5.4 仿真分析（或者结果分析）	23
六、问题四的模型建立与求解.....	24
6.1 问题分析.....	24
6.2 智能决策软件需求分析.....	24
6.3 智能决策软件总体设计.....	25
6.4 智能决策软件设计	25
6.4.1 前端用户界面功能结构	25
6.4.2 基础数据和输入参数	26
6.4.3 前端用户界面设计	27
6.4.4 后台决策程序功能结构	27
6.4.5 后台决策程序计算步骤	28
6.4.5 后台决策程序实现	29
七、模型评价与推广	29
7.1 模型的优缺点	29
7.2 模型推广	30
参考文献	31

附录 1: 问题一算法程序	31
附录 2: 问题二算法程序	38
附录 3: 问题三算法程序	70
附录 4: 问题四算法程序	84

一、问题重述

1.1 问题背景

2020 年我国城镇化率达到 63.89%，城镇化建设全面进入提质增效的存量更新阶段。2025 年 1 月 3 日，国务院召开常务会议研究推进城市更新工作，提出“城市更新关系城市面貌和居住品质的提升，是扩大内需的重要抓手”，并要求加快推进城镇老旧小区、街区、厂区和城中村等改造，保护和传承城市历史文化。

特大城市和大城市在经历快速发展历程后，其老城存量平房街区空间往往建筑产权多样、主体复杂，还涉及历史保护、居民社区治理等议题。近年来老城平房街区的更新实践往往是“一事一议”的渐进式、针灸式更新工作。在一事一议之下，居民搬迁已腾退出许多院落，可是在不少院落中，仍有居民不愿意接受“搬离老城、居民上楼”或者金额补偿，更愿意生活在本地，于是出现居民居住+腾空房屋共存一个院落的“共生”现象。这种现象为这些腾空房屋的再开发利用带来困难。为此，当前学界和业界提出一种新的策略——“平移置换”，此举既能保障居民居住在本地的诉求，又能够腾退置换出整院，以更好地实现院落再开发目标。然而，这项策略在老城街区的实施还存在亟需解决的搬迁问题。

以某城市某老街区为例，其内部存在 107 个大小不一的杂院，有一些彼此分离，有一些相互毗邻。每个杂院内可划分出不同的产权地块供人居住使用，共有 484 个适宜居住的地块，除去已经搬离的居民，仍有 113 户居民不愿意因为直接的金钱补偿搬离。如果要说服居民们搬家，应在搬迁后能提供更好的居住环境。在 2000 万搬迁投入（追加备用金限制不超过保底规划 30%）的保底规划下，拟采取面积补偿、采光补偿和修缮补偿方式推动居民同意搬迁。因为各杂院中的空置产权地块面积和朝向是确定且有限的，开发商不一定能为所有居民找到更合适的迁入地块，如果找不到，则该居民不搬迁。

开发商通常采取的盈利方式是出租给商户进行商业、旅游或办公功能的开发，规划局/开发商收取租金以获得收益。但如果仅开发单个院落中的零散房屋，避开尚在居住的居民，则不能实现如酒店、会馆、餐厅、咖啡厅等高价值功能，经济效益降低。从收益来看，直接出租分散的产权地块给开发商带来收入为：东西厢房屋 8 元/平米/天，南北厢房屋 15 元/平米/天。根据整院效益，完整的空置杂院为 30 元/平米/天。根据毗邻效益，空置毗邻整院的出租价值可增加为原来的 1.2 倍。从成本来看，开发商的搬迁投入包括搬迁沟通成本，修缮翻新费用，面积损失带来的经济损失，以及搬迁和开发的时间损失。

因此，有必要以科学的方式辅助规划局和开发商做出智能搬迁规划，给出带有“地块面积或住宿品质提升补偿策略”的平移置换规划，促进老房街区更新搬迁和开发利用。

1.2 问题提出

问题一：搬迁补偿建模

请根据当前情况，充分考虑住户房屋的朝向、面积、布局、心理价位、住宿舒适度等因素，设计合理的搬迁补偿方案，能根据住户情况给出合理的面积补偿、采光补偿和修缮补偿范围。除了文中提及的因素外，还有哪些因素可能会影响住户是否同意搬迁？作为规划局/开发商，应如何应对？请详细列出并定性/定量探讨（比如迁入迁出地块离街道的距离，地块周边的房屋密集程度等）。

问题二：整院面积最大

利用问题一的结果，设计针对每户住 户的搬迁决策，使得最终搬迁后空出的完整院落最多，在此基础上完 整院落尽可能相互毗邻，且总面积最大。这个过程中要注意搬迁住 户数应尽量少。请通过计算回答：应该如何搬迁？搬了哪些院子的居民， 从哪个地块搬到哪个地块，搬迁腾出的整院是哪些，并结算共投入多 少成本，最终的整院面积

和最终的收入及盈利（请注意尽量在保底规划成本内完成，如果实在不满足再使用备用金）。

问题三：性价增益的搬迁拐点计算

在不考虑保底规划成本约束时，计算投资回报率：开发商往往希望以十年为期限，使搬迁十年的租金收益相比于不搬迁的增量/实际搬迁投入（称为性价比 m ）大于等于 20。假定十年内租金没有明显变化，那么在搬迁进程中，是否存在性价增益的拐点？即达到某个搬迁的平稳状态，再搬迁带来的性价比开始变低了？如有，请计算出该拐点是哪种情况：搬了哪些院子的居民，从哪个地块搬到哪个地块，搬迁腾出的整院是哪些，并结算共投入多少成本，最终的整院面积和最终的收入及盈利。如没有，请讨论性价比 m 降低到至少多少就能存在拐点？

问题四：老城区平移置换决策软件设计

规划局希望这个案例能起到示范作用，推广到全国的老旧街区更新搬迁中，请你们设计一个软件框架，能够智能计算老城区平移置换决策，指出该软件应该设定哪些人工输入的参数，通过怎样的计算步骤，可以融合你们的模型设计，自动计算出问题二和问题三的答案。

二、模型假设与符号说明

2.1 模型假设

- 假设 1：假设附件中所给数据可靠并科学收集；
- 假设 2：居民不会从面积大的地块搬到面积小的地块，也不会从朝向评分高的地块搬到朝向评分低的地块；
- 假设 3：迁入地块的选择是一次性的，如果迁入后则不能换地块迁入；
- 假设 4：每个空置地块只能容纳一户居民，每个居民只能迁入一个地块；
- 假设 5：之前未迁出但选择迁出的地块不能被其他之前未迁出的住户迁入；
- 假设 6：租金、修缮成本、采光赋值等参数稳定不变，忽略市场波动、政策调整等动态因素，仅考虑一次性搬迁的调整需求。

2.2 符号说明

表 2.1 符号说明

No.	符号	问题一主要符号含义
1	I	院落集合， i 表示具体院落编号， $i \in 1, 2 \cdots N$ ， N 最大为 107
2	J	地块集合， j 表示具体地块编号， $j \in 1, 2 \cdots M$ ， M 最大为 484
3	J'	初始未入住的地块集合， j' 为未入住的地块
4	H	初始入住的地块集合， h 为初始有居民的地块
5	a_j	地块 j 的面积
6	G^h	地块 h 的最大搬迁补偿满意度
7	x_h	住户 h 是否搬迁，若搬迁则为 1，否则为 0， $\forall h \in H$

8	$x_{j'}^h$	地块 j' 是否可以在满足住户满意度的条件下供地块 h 迁入
9	$y_{j'}^h$	住户 h 是否搬迁入地块 j' , 若搬迁则为 1, 否则为 0, $\forall j' \in J'$
10	s_j	地块的采光系数值, 其中正南=正北=3, 正东=2, 正西=1
11	$\theta_{j'}$	j' 地块的拟修缮费用, 根据面积等因素综合给出
12	C_h^r	住户 h 搬迁修缮成本, $\forall h \in H$
13	$L_{ii'}^h$	地块 h 所在原院落 i 与搬入院落 i' 之间的距离
14	$\eta_{j'}^h$	住户所在地块 h 搬迁入地块 j' 后是否修缮房屋, 若修缮则为 1, 否则为 0, $\forall h \in H, j' \in J'$
15	τ_j	地块 j 所在院落中的空间句法综合指标, 包括深度、连接度与整合度等。其中 τ_j^1 表示整合度, τ_j^2 表示连接度, τ_j^3 表示步深
16	τ_{min}^1	与 j 同院落地块的最小整合度
17	τ_{max}^1	与 j 同院落地块的最大整合度
18	τ_{min}^2	与 j 同院落地块的最小连接度
19	τ_{max}^2	与 j 同院落地块的最大连接度
20	τ_{min}^3	与 j 同院落地块的最小步深
21	τ_{max}^3	与 j 同院落地块的最大步深
22	$\Delta D_{j'}^h$	迁入迁出地块距离最近街道的距离差
23	ΔD_{max}	迁入迁出地块的最大距离差为 ΔD_{max}
24	W	住户搬迁满意度

No.	符号	问题二主要符号含义
1	J'	空闲地块集合, 检索为 j'
2	A_i	院落 i 的总面积, $\forall i \in I$
3	a_j	地块 j 的面积, $\forall j \in J$
4	d_{ik}	院落 i 和院落 k 是否毗邻, 若毗邻则为 1, 不毗邻则为 0, $\forall i, k \in I$
5	s_j	地块 j 的采光条件, $\forall j \in J$
6	T	出租/开发天数

7	v_j	地块 j 的每日出租/开发价值, $\forall j \in J$
8	θ_j	地块 j 的修缮成本, $\forall j \in J$
9	C_h^m	居民户 h 搬迁沟通成本, $\forall h \in H$
10	C_h^a	居民户 h 搬迁面积损失成本, $\forall h \in H$
11	C_h^t	居民户 h 搬迁时间损失成本, $\forall h \in H$
12	z_i	院落 i 是否为整院, 若为整院则为 1, 否则为 0, $\forall i \in I$
13	w_{ik}	院落 i 和院落 k 是否为整院且毗邻, 若为整院且毗邻则为 1, 否则为 0, $\forall i, k \in I$
14	$\eta_{j'}^h$	居民户 h 搬迁入地块 j' 后是否修缮房屋, 若修缮则为 1, 否则为 0, $\forall h \in H, j' \in J'$

No.	符号	问题三主要符号含义
1	m	性价比指标
2	m_{min}	性价比指标最低阈值
3	R_0	不搬迁的十年租金收益
4	R_1	搬迁后的十年租金收益
5	z_i^0	初始不搬迁时院落 i 是否为整院院落, 若为整院则为 1, 否则为 0, $\forall i \in I$
6	w_{ik}^0	初始不搬迁时院落 i 与院落 k 是否为毗邻整院, 若为毗邻整院则为 1, 否则为 0, $\forall i, k \in I$
7	v_j^0	初始不搬迁时地块 j 的出租/开发价值, $\forall j \in J$
8	u_j	地块 j 是否空置, 若空置则为 1, 否则为 0, $\forall j \in J$

No.	符号	问题四主要符号含义
1	m_0	性价比期望下限

2.3 数据说明

题中共给出两个附件, 附件一为老城街区地块信息, 包含属于老城街区的 484 个地块的相关基础信息: 地块 ID、地块面积、地块朝向、所属的院落 ID、院落面积以及地块上是否有居民居住 (1 表示有人居住, 0 表示空闲)。该矩阵作为所有问题的基础数据十分重要, 需要将地块 ID 与各种地块基本信息进行关联以便于后续的调用求解, 详细数据读取与处理见附录一部分代码。

附件二为老城街区的地块显示，在图中可以获得两个地块是否毗邻的信息，进而生成院落之间的毗邻矩阵，便于第二问与第三问对院落关系进行处理。同时，还能初步获取各院落的分布位置、与街道的距离及各院落内的地块布局，便于问题一进行计算空间句法综合指标等。

三、问题一的模型建立与求解

3.1 问题分析

问题一的核心在于建立搬迁补偿模型用于综合评估每一住户的搬迁在现有条件下接受平移置换搬迁的意愿。所建立的模型综合考虑面积、朝向、布局、心理价位、住宿舒适度等因素，设定面积补偿、采光补偿和修缮补偿为主要指标。此外，根据其辅助影响因素，如迁入迁出地块离街道的距离，地块周边的房屋密集程度等进一步完善评价模型。同时，基于面积补偿、采光补偿和修缮补偿范围，确定使用户搬迁意愿达到最大的搬迁补偿方案的约束条件。最后，从规划局/开发商的角度给出除文中提及因素的其他影响因素的应对策略。

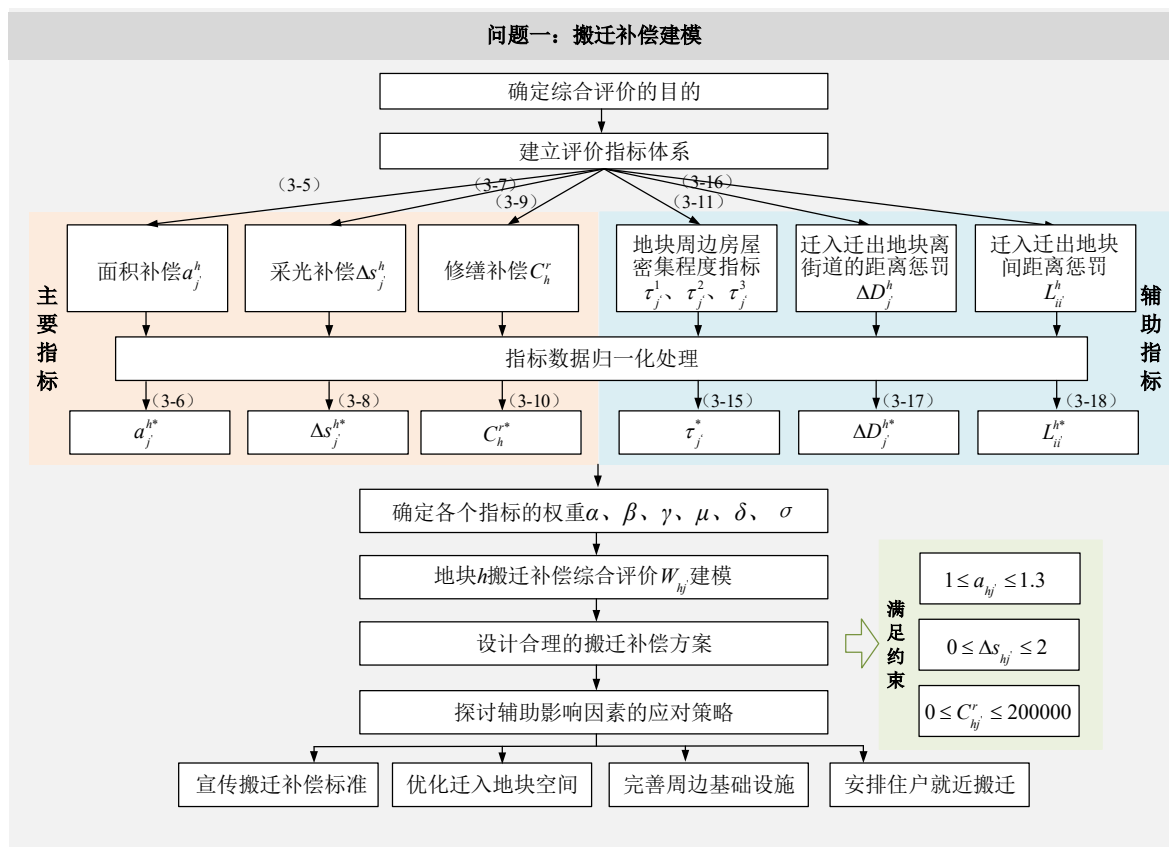


图 3.1 问题一搬迁补偿建模流程图

3.2 建立居民搬迁意愿评价模型

3.2.1 搬迁补偿满意度模型

定义从未搬地块 h 向已搬离地块 j' 搬迁的住户满意度模型（搬迁补偿模型）：

$$W_{j'}^h = \alpha a_{j'}^{h*} + \beta \Delta s_{j'}^{h*} + \gamma C_h^{r*} + \mu \tau_{j'}^* - \delta \Delta D_{j'}^{h*} - \sigma L_{ii}^{h*} \quad (3-1)$$

其中， α 、 β 、 γ 、 μ 、 δ 、 σ 分别为面积补偿、采光补偿、修缮补偿、空间句法综合指标、迁入迁出离街道的距离惩罚和搬迁距离惩罚的权重，满足 $\alpha + \beta + \gamma + \mu + \delta + \sigma = 1$ 。由于住户

满意度受个体差异的影响比较明显,权重可以根据民意调查法或专家评分法进行确定^[1]。

建模的目标是最大化未搬地块 h 住户的满意度,即

$$G_h = \max(W_{j'}^h | j' \in J') \quad (3-2)$$

通过比较满意度阈值 G_{th} 和最大满意度 G^h 来判断住户是否愿意搬迁,即

$$x_h = \begin{cases} 1, G_h \geq G_{th} \\ 0, G_h < G_{th} \end{cases} \quad (3-3)$$

若最大满意度大于阈值,则继续判断 J' 中其他地块是否大于阈值

$$x_{j'}^h = \begin{cases} 1, W_{hj'}^h \geq G_{th} \\ 0, W_{hj'}^h < G_{th} \end{cases} \quad (3-4)$$

通过以上判断可以筛选出所有满足地块 h 的迁入地块集合。

3.2.2 评价指标及无量纲化

(1) 面积补偿约束

根据题意,如果住户搬迁,则待搬入的地块面积 a_j 应大于等于原来的面积 a_h ,并且不超过原面积的 1.3 倍,因此搬入地块面积满足以下约束

$$1 \leq a_{j'}^h = y_{j'}^h \cdot \frac{a_{j'}}{a_h} \leq 1.3 \quad (3-5)$$

作为搬迁补偿的一种形式,面积补偿将会对住户搬迁产生积极推动作用。这里,采用计算搬迁后的面积变化率来建立模型。为了统一量纲,对 $a_{j'}^h$ 除以最大面积补偿率 1.3 得到归一化面积补偿指标如下:

$$a_{j'}^{h*} = a_{j'}^h / 1.3 \quad (3-6)$$

(2) 采光补偿约束

根据题意,如果住户搬迁,则待搬入地块的采光效果 $s_{j'}$ 不小于原来的采光效果 s_h 。这里,采用采光系数值定量衡量采光效果,即南=北=3,东=2,西=1,则搬迁补偿需要满足以下约束:

$$0 \leq \Delta s_{j'}^h = y_{j'}^h (s_{j'} - s_h) \leq 2 \quad (3-7)$$

通过对除以最大采光系数变化 2,得到归一化的采光补偿指标,即

$$\Delta s_{j'}^{h*} = \Delta s_{j'}^h / 2 \quad (3-8)$$

(3) 修缮补偿约束

当搬入地块的采光系数与面积都与原来保持一致时,为了提高居民的搬迁意愿,需要付出一定的修缮补偿。这里,房屋面积越大,接受面积和采光补偿的意愿可能就越低,因此,假设补偿金额与房屋面积有关。对于最大房屋面积 237 平米,其最大修缮资金不超过 200000,用 20000 除以最大的房屋面积 237 可以近似每平米的修缮资金补偿,再乘以待搬入的地块面积数即可代替这种搬迁方案的修缮资金补偿,计算公式如下:

$$C_h^r = \sum_{j' \in J'} y_{j'}^h \cdot \eta_{j'}^h \cdot \frac{200000}{237} \cdot a_h \quad (3-9)$$

其中,当 $y_{j'}^h=1$, $s_h=s_{j'}$ 且 $a_h=a_{j'}$ 时 $\eta_{j'}^h=1$, 否则为 0。

同理,通过将 C_h^r 除以最大修缮资金补偿的方法实现归一化,即

$$C_h^{r*} = C_h^r / 200000 \quad (3-10)$$

(4) 地块周边房屋密集程度评估指标

空间句法是比尔希列尔在 20 世纪 70 年代 提出的一套量化研究空间的理论和方法。根据这一理论，建筑空间应被看作具有特定形态和结构的系统，而非简单的元素堆砌。空间句法将空间作为一个整体，用图形符号来描述环境的平面布局，并通过分析空间的组织关系来探讨其构成。国际上，研究空间句法的团队开发了 Depthmap 软件，该软件使用定量计算方法将建筑内部空间形态转化为数据，帮助建筑学者深入研究其内涵^[2]。本题中，引入空间句法中的整合度，连接度和步深的概念进行地块周边房屋密集程度指标的量化。

① 整合度

整合度表示衡量该房间在整个房屋中的中心性（基于最短路径计算），数值越高表示空间的可达性越高，周围空间布局较为集中。以房内布局为例，客厅到其他房间的平均步数较小，代表高整合度，而储藏室位于角落，平均步数较大，代表低整合度。标准化全局整合度的计算方式如下：

$$\tau_{j'}^1 = \frac{1}{meandepth_j} \quad (3-11)$$

其中， $meandepth_j$ 是该房间到所有其他房间的平均步数。

② 连接度

连接度表示某房间能直接连接其他房间的个数，能够体现某房间对院落其他房间的可达性。文中用 τ_j^2 表示。

③ 步深

步深表示从房屋主入口到该房间的最短步数（拓扑距离），反应了房屋对外交流的便捷程度，用 τ_j^3 表示。

以题中附件所给的老街地块图示的院落 9 为例，在 AUTOCAD 中绘制院落结构并在 depthmap 软件中对院落内地块的三项空间句法指标进行分析，结果如下：

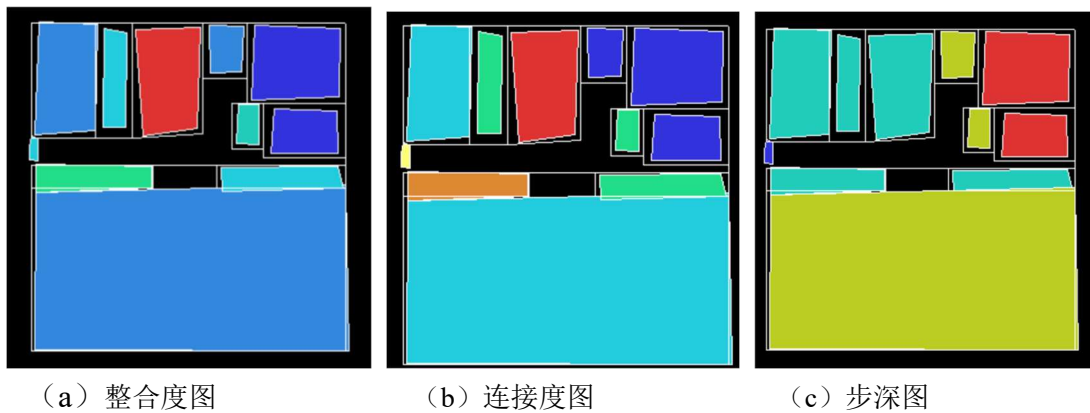


图 3.2 空间句法各指标含义示意图

如上图所示，由深蓝到红色表示整合度、连接度、步深深逐渐提高，可见离门最远的右上角地块整合度与连接度都偏低，并且步深高，表示可达性差，与院落较为割裂，可能会影响居民对地块的选择。

由于三者的量纲不同，需进行归一化处理，归一化步骤如下：

$$\overline{\tau_{j'}^1} = \frac{\tau_j^1 - \tau_{min}^1}{\tau_{max}^1 - \tau_{min}^1} \quad (3-12)$$

$$\overline{\tau_{j'}^2} = \frac{\tau_j^2 - \tau_{min}^2}{\tau_{max}^2 - \tau_{min}^2} \quad (3-23)$$

$$\overline{\tau_{j'}^3} = 1 - \frac{\tau_j^3 - \tau_{min}^3}{\tau_{max}^3 - \tau_{min}^3} \quad (3-34)$$

④ 空间句法总指标

根据现实需要与居民对某项指标的看重程度动态调节三个指标的权重系数。

$$\tau_{j'}^* = \mu_1 \bar{\tau}_j^1 + \mu_2 \bar{\tau}_j^2 + \mu_3 \bar{\tau}_j^3 \quad (3-45)$$

其中, $\mu_1 + \mu_2 + \mu_3 = 1$

(5) 迁入迁出地块离街道的距离惩罚

当迁入迁出地块距离街道的距离会影响位置便利度, 进而影响搬迁意愿。因此, 引入迁入迁出搬迁距离惩罚, 定义迁入迁出地块距离最近街道的距离分别为 $D_{j'}$ 和 D_h 满足迁入距离不小于迁出距离的约束, 即

$$\Delta D_{j'}^h = y_{hj'}(D_{j'} - D_h) \geq 0 \quad (3-56)$$

归一化有:

$$\Delta D_{j'}^{h*} = \Delta D_{j'}^h / \Delta D_{\max} \quad (3-67)$$

(6) 迁入迁出地块间距离惩罚

当迁入迁出之间的距离过远时, 会影响居民的搬迁意愿, 因此引入搬迁距离惩罚, 考虑搬迁前后的院落之间的距离 $L_{ii'}^h$, 定义最大距离为 $L_{\max}$, 对该指标进行归一化可知:

$$L_{ii'}^{h*} = L_{ii'}^h / L_{\max} \quad (3-78)$$

式中, $h \in i, j' \in i', i' \neq i$ 。

3.3 补偿应对策略

规划局是政府部门的职能部门, 负责城市规划、土地使用、建设审批等工作, 确保在多方共赢的情况下, 有质有量地推动改造工程的完成。开发商是老旧街区改造的执行方和运营方, 以赚取最多利益为目标, 但是单方面的利益最大化, 并不可能持续, 应适当平衡租金利益与社会效益之间的关系^[3], 提高搬迁决策的吸引力。因此, 两者的协调配合对于提高住户搬迁率具有很大影响。针对地块周边的房屋密集程度、迁入迁出地块离街道的距离和迁入迁出地块间距离, 主要提出以下几点建议如下:

(1) 宣传搬迁补偿标准

为了让住户深入了解搬迁目的、实施计划、预期效益及其对城市发展的积极影响, 通过社交媒体等多种途径发布政策文件、公告等, 解释城市规划、土地使用政策等, 提高住户对城市发展的认知, 增强规划政策的透明度。同时, 将搬迁补偿标准、老旧城区的改造、迁入地区基础设施建设通过座谈会、走访等形式解释到位, 增强住户认同感, 避免由于信息不通造成的不理解、不支持。

(2) 优化迁入地块空间

地块周边房屋的密集程度会影响住户居住舒适度。为了减少房屋过于密集所造成的舒适度损失, 规划局或开发商有必要从优化迁入地块居住环境的角度出发采取补偿行动。一是增加绿化空间, 在迁入地块设置绿化带、人工水景等营造舒适美观的空间, 分散居民的活动密度。二是设计如运动场、图书馆、花园等共享空间, 从视觉和心理上扩大活动空间, 提高空间利用率。

(3) 完善周边基础设施

调查显示, 区位是决定搬迁住户满意度的重要因素之一^[4], 而迁入迁出地块区位的变化会影响交通通勤的便利性和公共设施的可达性。为了提高交通条件, 设置公交车站、共享单车、停车位, 并优化路网连接度和交通接驳便利性, 为居民提供良好的出行体验, 弥补区位优势降低。此外, 在迁入地区周边规划公共设施的建设, 如医院、商场、商

业街、教育机构等，满足住户的多样化需求，提高住户满意度。

（4）安排住户就近搬迁

迁入迁出地块间距离在一定程度上会导致满意度的降低。为提高满意度，一方面，在规划阶段，通过合理安排搬迁地块的布局，尽量让住户在同一地区或相邻区域内进行搬迁，避免长距离搬迁，减少不必要的搬迁成本。另一方面，在搬迁阶段，安排搬迁车辆或提供搬迁补贴，如与网约车平台合作发放优惠券，确保住户便捷迁入新居，减轻额外搬迁成本。

3.4 模型求解

Yalmip 是一个 MATLAB 的工具箱，主要用于动态规划模型的快速、简单建模。在模型求解过程中，除了 Yalmip 内部提供的基础线性规划算法可以求解外，还可以调用外部最优化求解器（如 Gurobi, CPLEX 等）进行求解^[5]。用户只需将构建的数学模型以规定形式输入，并通过配置参数指定求解方法，YALMIP 就会自动将其转换成求解器可识别的语言格式进行求解。Yalmip 允许调整模型中的某些参数，并可以对这些参数进行灵敏度分析。

Gurobi 优化器是一款强大的数学规划求解器，它为线性规划、混合整数线性规划、二次规划等问题的解决提供了可靠且快速的求解策略^[6]。由于其快速的求解速度和优质的求解结果，在解决动态规划和一般优化问题中受到广泛应用。在求解器领域，Gurobi 在处理多变量约束的复杂大规模问题方面通常优于其他求解器。此外，它在并行计算能力、高级算法、多核 CPU 的优化性能等方面都具有突出优势。

为了提高所建立的混合整数线性规划（MILP）模型的求解效率，本文基于 Matlab 的 YALMIP 工具箱，在求解过程中调用 Gurobi 求解器实现。

四、问题二的模型建立与求解

4.1 问题分析

某老城街区的院落情况如图 所示，共有 107 个院落，黑色方框范围为院落轮廓，编号用绿色数字标记。院落中共有 484 各地块，其中深红色块状为已居住地块，浅红色地块为暂未居住地块。老城街区改造的核心是在有限保底规划成本内，通过科学搬迁决策实现最大化空出的整院数量的主要目标以及整院尽可能毗邻且面积最大、搬迁住户数尽可能少等次要目标。



图 4.1 某老城街区“共生”院落位置、编号及毗邻情况

本题针对老城街区地块开发中的经济效益优化问题，构建了一个混合整数线性规划（MILP）模型。模型以最大化规划局/开发商的净收益为目标，综合考虑整院开发收益、毗邻效应加成、搬迁成本及多类资源约束，旨在为城市更新项目提供科学决策支持。报告详细阐述了模型的参数体系、目标函数、约束条件及求解策略，并结合搬迁补偿相关理论与实际案例进行了分析。

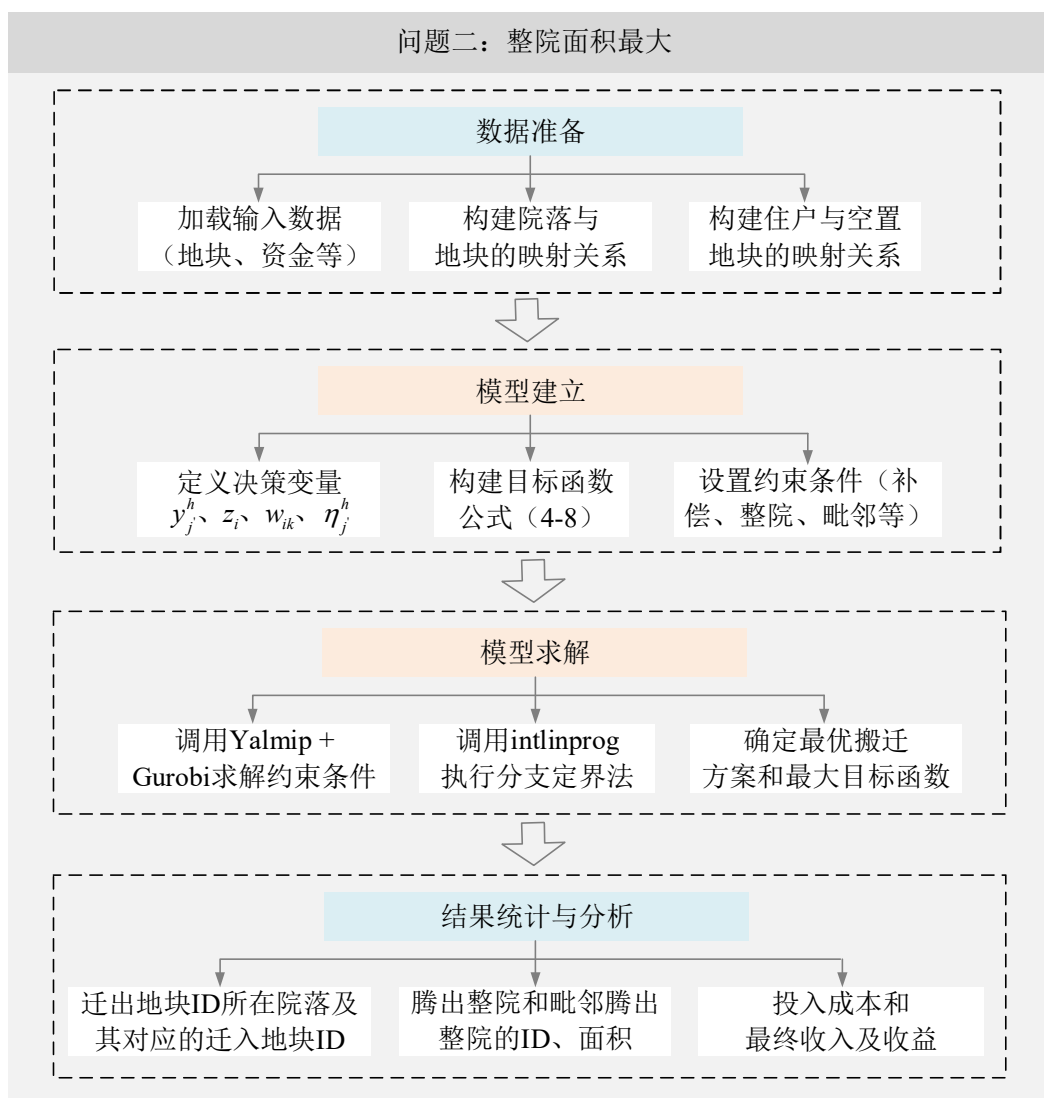


图 4.2 问题二整院面积最大的建模与求解

4.2 建立整院数量最大化与毗邻效益协同优化模型

混合整数线性规划（MILP）是优化问题的一种建模方式，包括目标函数、约束条件和决策变量。MILP 结合了线性规划和整数规划的特点，其目标函数和约束条件都是线性的，并且包含整数变量。通过优化 MILP 模型的目标函数，能够处理具有复杂约束的问题，并给出成本、效率、利益等的规划结果。因此，MILP 在实际应用中广泛应用于生产调度、网络设计、资源分配等。

4.2.1 决策变量

为对所规划问题进行建模，主要设置以下决策变量：

- (1) 决策变量 y_j^h ，用于判断居民户 h 是否搬迁入地块 j' ，若搬迁则为 1，否则为 0， $\forall h \in H, j' \in J'$
- (2) 决策变量 z_i ，用于判断院落 i 是否为整院，若为整院则为 1，否则为 0， $\forall i \in I$;

(3) 决策变量 w_{ik} , 用于判断院落 i 和院落 k 是否为整院且毗邻, 若为整院且毗邻则为 1, 否则为 0, $\forall i, k \in I$;

(4) 决策变量 η_j^h , 用于判断居民户 h 搬迁入地块 j' 后是否修缮房屋, 若修缮则为 1, 否则为 0, $\forall h \in H, j' \in J$ 。

4.2.2 目标函数

由题意可知, 目标函数的构建可以通过最大化规划局/开发商的经济效益, 来实现整院院落数量及整院面积最大化, 并在此基础上尽可能满足相互毗邻的整院院落最大和搬迁住户数最小。规划局/开发商的整院经济效益可以通过整院租金收益减去搬迁成本获得。假设开发周期为 T 月, 分别计算租金收益和搬迁成本。

租金收益的来源包括不考虑毗邻效益的整院收益和整院毗邻的额外收益。已知整院租金为 30 元/ m^2 /天、毗邻整院额外获得 20% 的收益, 则租金收益计算如下:

$$Z_1 = \sum_{i \in I} T \times 30 \cdot A_i \cdot z_i \quad (4-8)$$

$$Z_2 = \sum_{i, k \in I} T \times 30 \cdot (A_i + A_k) \cdot 0.5 \cdot 0.2 w_{ik} \quad (4-2)$$

Z_1 表示不考虑毗邻效益的整院收益, Z_2 表示整院毗邻的额外收益。

搬迁成本来源包括搬迁沟通成本, 修缮翻新费用, 面积损失带来的经济损失, 以及搬迁和开发的时间损失。已知每腾出一个地块的沟通成本为 3 万元, 修缮成本上限 20 万元 (可以不修缮), 面积经济损失折算为多补偿给居民的面积潜在租金, 时间损失折算为搬迁耗时 4 个月的出租收益损失。四种成本分别计算如下:

$$C_h^m = 30000 \sum_{j' \in J'} y_{j'}^h, \forall h \in H \quad (4-3)$$

$$C_h^a = \sum_{j' \in J'} v_{j'} \cdot (a_{j'} - a_h) \cdot y_{j'}^h \cdot T, \forall h \in H \quad (4-4)$$

$$C_h^r = \sum_{j' \in J'} \theta_{j'} \cdot y_{j'}^h \cdot \eta_{j'}^h, \forall h \in H \quad (4-5)$$

$$C_h^t = \sum_{j' \in J'} (v_h \cdot a_h + v_{j'} \cdot a_{j'}) \cdot y_{j'}^h \cdot 120, \forall h \in H \quad (4-6)$$

其中, 修缮成本 θ_j 与迁入面积成正比的变量。开发商要求最大修缮补偿不能超过 20 万元, 为了使得修缮成本定量化, 将修缮成本与地块的面积相关联, 即面积越大所需的修缮资金越多[1]。为了保证每户的修缮补偿不会超过 20 万元并保证补偿的公平性[2], 找出最大地块面积, 假设其搬迁所能接收的最大补偿为 20 万元, 计算每平米的修缮资金, 最后以面积比例来量化每个地块的修缮成本, 得到 θ_j 的计算公式为

$$\theta_j = 200000 \cdot \frac{a_j}{237}, \forall j \in J \quad (4-7)$$

综合租金收益和搬迁成本确定的目标函数如下

$$\begin{aligned} \max Z = & \sum_{i \in I} T \times 30 \cdot A_i \cdot z_i + \sum_{i, k \in I} T \times 30 \cdot (A_i + A_k) \cdot 0.5 \cdot 0.2 w_{ik} \\ & - \sum_{h \in H} (C_h^m + C_h^a + C_h^r + C_h^t) \end{aligned} \quad (4-8)$$

4.2.3 约束条件

(1) 面积补偿约束

每户搬迁居民到新地块的面积不能小于原面积, 并且不超过原面积的 130%, 约束如下

$$a_h \leq a_{j'} \leq 1.3 a_h, \forall h \in H, j' \in J', y_{j'}^h = 1 \quad (4-9)$$

(2) 采光补偿约束

搬迁后采光条件不能弱于原采光条件，约束如下

$$s_h \leq s_{j'}, \forall h \in H, j' \in J', y_{j'}^h = 1 \quad (4-10)$$

(3) 修缮成本约束

修缮补偿作为开发商对那些对待搬入地块的面积和朝向都不满足心理预期而使用的资金，本文用当搬迁前后面积与采光系数都相等时启用资金作为修缮补偿的条件。同时，修缮补偿作为开发商对那些对待搬入地块的面积和朝向都不满足心理预期而使用的资金，本文用当搬迁前后面积与采光系数都相等时启用资金作为修缮补偿的条件，约束如公式（4-11）-（4-13）所示。

$$\eta_{j'}^h = \max\{1 - M(a_{j'} - a_h) - M(s_{j'} - s_h), 0\}, \forall h \in H, j' \in J', y_{j'}^h = 1 \quad (4-11)$$

$$\eta_{j'}^h \leq y_{j'}^h, \forall h \in H, j' \in J' \quad (4-12)$$

$$v_j = \begin{cases} 15, s_j = 3, z_i = 0 \\ 8, s_j = 1, s_j = 2, z_i = 0 \\ 30, \forall s_j, z_i = 1 \end{cases} \quad (4-13)$$

其中 M 为足够大的数。

(4) 地块容量约束

地块可容纳居民户容量表示一个搬入地块只允许最多搬入一个居民户，约束公式为：

$$\sum_{h \in H} y_{j'}^h \leq 1, \forall j' \in J' \quad (4-14)$$

(5) 搬迁成本约束

成本约束可以分为保底规划成本与启用备用金的两种情况，分别由公式（4-15）和公式（4-16）

$$\sum_{h \in H} (C_h^m + C_h^a + C_h^r + C_h^t) \leq 20000000 \quad (4-15)$$

$$\sum_{h \in H} (C_h^m + C_h^a + C_h^r + C_h^t) \leq 26000000 \quad (4-16)$$

(6) 完整院落约束

根据题意，只有当院落 i 内所有居民搬迁后才能整院出租，即只有当院落内所有居民搬迁（每户 $\sum(j' \in J') y_{j'}^h = 1$ ），院落才有可能设定为整院出租，该约束可以写成以下形式：

$$z_i \leq \sum_{j' \in J'} y_{j'}^h, \forall h \in H_i, i \in I \quad (4-17)$$

根据公式（4-17），求和部分代表院落 i 的地块集合 H_i 中的任一地块 h 在所有空地集合 J' 中选择理想迁入地块 j' 并赋值为 1，而地块集合中的其他地块赋值为 0。此时，地块 h 的求和结果始终为 1， $z_i \leq 1$ ，说明 z_i 有可能为 1。若 H_i 中所有地块的求和结果都为 1，说明院落 i 有可能已经腾空为完整院落，即 z_i 可能为 1。反之，如果院落地块集合 H_i 中有任一地块的住户未迁出， J' 中所有地块赋值为 0，地块 h 的求和结果为 0， $z_i \leq 0$ ，这时 z_i 只能等于 0，说明该院落不可能是空院落。因此，公式（4-17）可以初步排除仍有居民未搬迁的院落，并确定整院决策变量 z_i 的上界。

当利用公式（4-17）判定院落 i 的 $z_i \leq 1$ 时，为筛选所有住户 $h \in H_i$ 均被搬迁的整院或院落原本为空院落的整院，需要采用公式（4-18）进一步约束。

$$z_i \geq \sum_{h \in H_i} \sum_{j' \in J'} y_{j'}^h - (|H_i| - 1), \forall i \in I \quad (4-18)$$

其中， $|\cdot|$ 表示取模运算。公式（4-18）右侧求和部分表示院落 i 所有待搬出地块集合 H_i 迁出决策变量的求和，如果 H_i 所有地块都选择迁出，则此时求和部分的值等于 $|H_i|$ ，再减去 $|H_i|-1$ 得到 $z_i \geq 1$ ，院落被标记为整院（ $z_i=1$ ）。此外，如果院落原本为空院落，则右侧求和部分等于 0，再减去 $|H_i|-1$ 仍然可得 $z_i \geq 1$ ，同样可以被标记为 $z_i=1$ 。反之，如果

院落 i 中有小于 $|H_i|$ 的地块搬迁, 则会导致 z_i 大于等于一个不大于 0 的数, 此时 $z_i=0$, 此时院落 i 不能认定为整院。因此, 公式 (4-18) 可以确定所有整院, 并确定整院决策变量 z_i 的下界。

(7) 毗邻关系约束

院落 i 和院落 k 同时满足整院空置且毗邻时, 才能获得毗邻额外收益, 因此, 需要确定毗邻的约束条件。假设 i 和 k 是两个相互毗邻的院落, 则毗邻整院的决策变量需要大于等于两个院落整院决策变量的加和减去 1, 并且毗邻整院的决策变量不大于任一整院的决策变量和两个院落的毗邻变量。此外, 若院落编号相同, 则不能表示毗邻整院关系。约束公式表示如下:

$$w_{ik} \geq z_i + z_k - 1, \forall i, k \in I, d_{ik} = 1 \quad (4-19)$$

$$w_{ik} \leq z_i, \forall i, k \in I \quad (4-20)$$

$$w_{ik} \leq z_k, \forall i, k \in I \quad (4-21)$$

$$w_{ik} \leq d_{ik}, \forall i, k \in I \quad (4-22)$$

$$w_{ii} = 0, \forall i \in I \quad (4-23)$$

若考虑保底规划成本约束, 该问题优化模型为 (4-1) - (4-15)、(4-17) - (4-23); 若考虑使用备用金, 该问题优化模型为 (4-1) - (4-14)、(4-16) - (4-23)。

4.3 模型求解

分支定界法是一种通过搜索与迭代进行规划求解的方法, 在规划和优化组合的问题上具有重要作用^[7]。该方法通过选择不同的分支变量和子问题进行分支。在每次分支后, 超出已知可行解集目标值的子集不能进行再分支, 等效为剪。通过分支和剪支操作, 搜索范围逐渐缩小, 最终可以获得最优解。

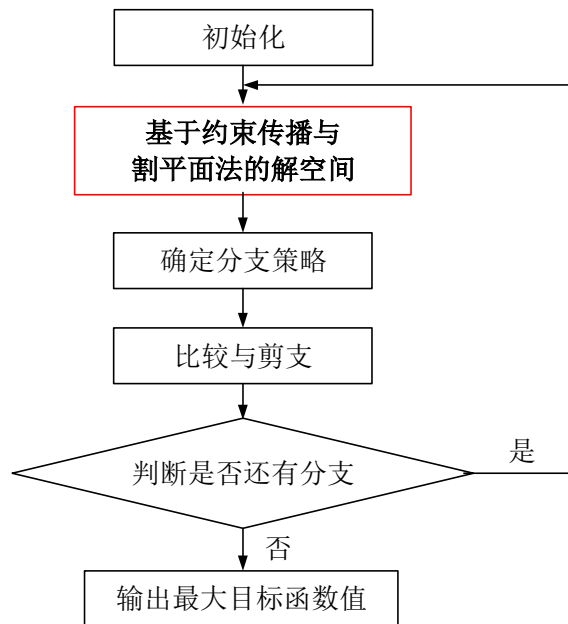


图 4.3 分支定界法求解思路

本文采用分支定界法搜索所建立的混合整数规划模型的最优解。求解步骤如下:

Step 1: 输入目标函数、约束矩阵和变量边界, 并据此产生初始模糊解。

Step 2: 结合约束传播与割平面法缩小解空间, 加速搜索速度。

Step 3: 在缩小解空间的基础上确定最大伪成本规则, 优先处理对目标函数影响最大的变量, 并将变量进行分支得到子问题, 确定分支策略。

Step 4: 通过比较, 对不可行的子问题解, 或松弛解的目标值大于等于当前子问题求

出来的目标值，或不满足约束条件的子问题解进行剪支。

Step 5: 判断是否还能进行分支，若为否，则输出最优解和最大目标值，若为是，则进入下一次循环。

4.4 结果分析

在对问题二建立数学模型以及设计算法求解步骤之后，通过代码运行结果可以得出问题二优化模型求解后的数据，得到设计搬迁决策如表 4.1，搬迁后地块院落信息如表 4.2，搬迁后投入成本数据如表 4.3、4.5，搬迁后租金收益及盈利如表 4.4、4.6，这里设定搬迁一年和十年后的数据如下：

表 4.1 搬迁的住户 OD 表

搬迁前 ID	搬迁后 ID	搬迁前 ID	搬迁后 ID
14	357	235	257
91	19	266	143
92	2	267	466
93	281	268	470
110	4	296	170
111	119	297	136
138	173	327	214
139	247	380	51
140	277	381	123
141	183	384	213
142	229	385	399
179	290	393	422
180	300	409	78
181	58	424	70
182	341	433	387
200	426	459	402
203	209	461	463
204	130	462	64
215	115	468	273
224	83	469	124
225	467		

表 4.2 搬迁地块-院落数据结果表

数据名称	数据结果
总搬迁住户数	41 个
完全腾退院落数量	23 个
腾退院落的总面积	9663.00 m ²

完全腾退院落 ID	[5,20,26,31,42,45,46,49,51,52,58,64,74,86, 87,89,91,95,98,99,104,105,106]
空置院落总面积	29526 m ²

表 4.3 搬迁投入成本数据结果表（以搬迁一年计）

数据名称	数据结果
沟通成本	1230000 元
修缮成本	1050000 元
面积损失成本	1505880 元
时间损失成本	6899160 元
投入总成本	10685040 元

表 4.4 搬迁投入成本数据结果表（以搬迁十年计）

数据名称	数据结果
沟通成本	1230000 元
修缮成本	1050000 元
面积损失成本	15058800 元
时间损失成本	6899160 元
投入总成本	24237960 元

表 4.5 搬迁租金收益数据结果表（以搬迁一年计）

数据名称	数据结果
整院租金收益	280068150 元
毗邻院落额外租金收益	42054570 元
分散空置地块租金收益	18660990 元
年租金总收入	340783710 元
搬迁一年后盈利	330098670 元

表 4.6 搬迁租金收益数据结果表（以搬迁十年计）

数据名称	数据结果
整院租金收益	2800681500 元
毗邻院落额外租金收益	420545700 元
分散空置地块租金收益	186609900 元
年租金总收入	3407837100 元
搬迁十年后盈利	3383599140 元

通过求解出来的表 4.1、4.2、4.3、4.4 内的数据，可以得到搬迁方案，即问题二中“应该如何搬迁，搬了哪些院子的居民，从哪个地块搬到哪个地块”这一具体方案，得到搬迁腾出的整院 ID 集合，最终以搬迁一年即 $T=365$ 结算出搬迁一年的投入总成本为 10685040 元，由于面积损失成本的计算包含时间因素，故面积损失成本随着时间动态变

化，投入总成本也会变化。此处以一年为例得出具体数值，搬迁一年租金总收入为 340783710 元，共计盈利 330098670 元，并且搬迁决策在保底规划成本内完成。

下面对结果进行可视化分析，三张图片分别从空间分布、腾退院落定位、毗邻收益关联三个维度，直观呈现了搬迁决策方案的实施效果。如图 4.4 所示，该图展示了老城街区中腾退地块（红色）与非腾退地块（蓝色）的空间分布。 X 轴表示地块的横坐标维度， Y 轴表示地块的纵坐标维度，腾退地块（红色）集中在 X 坐标 0-1000、 Y 坐标 1、2、3 的区域，表明这些区域是搬迁后空置的主要区域，可能用于高价值商业开发，实现如酒店、会馆、餐厅、咖啡厅等高价值功能。非腾退地块（蓝色）分散在周边，多为仍存留住户的地块，符合模型结果中“共生院”现象的描述。

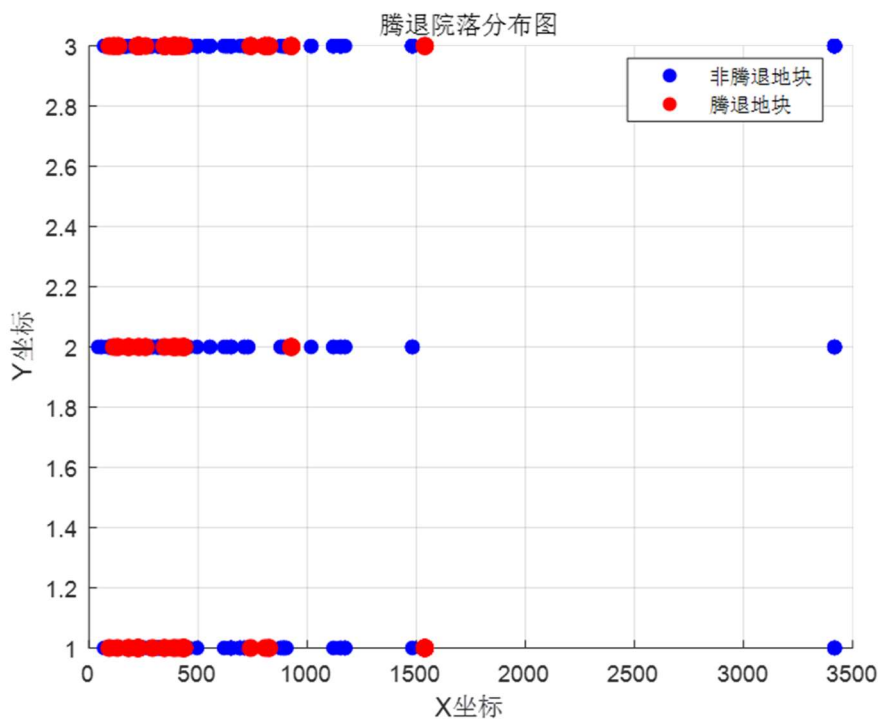


图 4.4 腾退院落分布可视化图

图 4.5 以毗邻腾退关系网络节点形式标注了搬迁后完全腾退的院落位置，红色节点代表腾退院落，其余为未腾退或部分腾退院落。红色节点集中在街区如院落 ID 86、87、89、91、95、104、105 等，与表 4.2 中“完全腾退院落 ID 列表”一致，节点密集，表明该区域毗邻整院较多，可能形成高收益的毗邻组合。腾退院落多沿主干道分布，其空间特征符合“毗邻效益最大化”的规划目标，从获益角度看，相互毗邻的空置整院越多即整院节点越密集，将为开发商带来越大的经济效益。

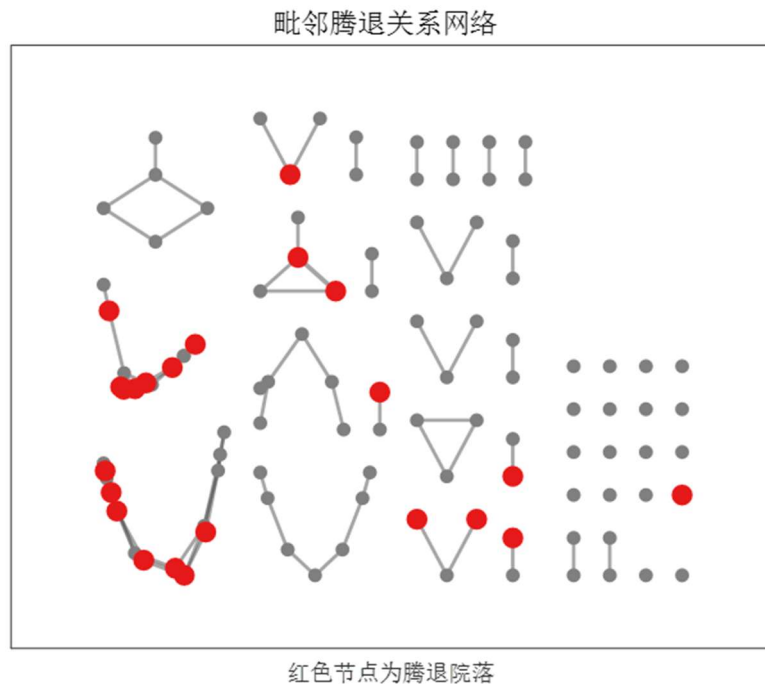


图 4.5 毗邻腾退关系网络节点可视化图

图 4.6 展示了通过红色边连接了产生额外收益的毗邻整院对，直观展示搬迁后形成的毗邻整院组合。如院落 106 与 5、46 与 45、51 与 49、99 与 98 等，这些院落对满足“整院且毗邻”条件，可享受除整院外的 20% 的额外租金加成。而未通过红色边连接的院落则视为整院但不相互毗邻，只可享受整院的租金加成，如院落 86、95 等。

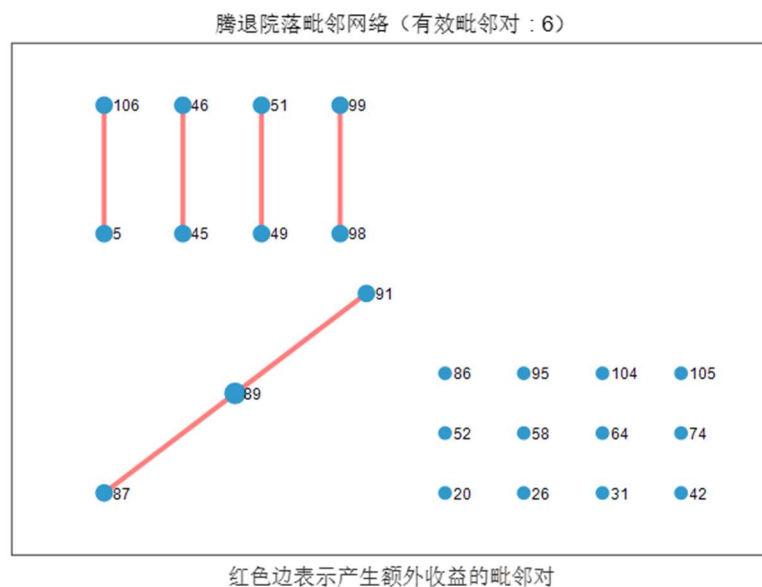


图 4.6 腾退院落毗邻可视化示意图

图 4.7 以附件中给出的数据对搬迁决策方案实施后所得出的腾退院落面积进行排序，得到所有腾退院落面积排序柱状图，可以从图中看出，院落 98 的腾退面积最大，从获益角度看，腾出的整院子面积越大，出租效益越好，故开发商在说服居民们搬家且一次性解决搬迁问题以及为获得更大的经济效益时，尽可能首先考虑腾退整院面

积最大的院落。图 4.8 展示了搬迁前后空置院落数量增加这一结论，满足题设要求“最终搬迁后空出的完整院落最多，总面积最大”这一结论。

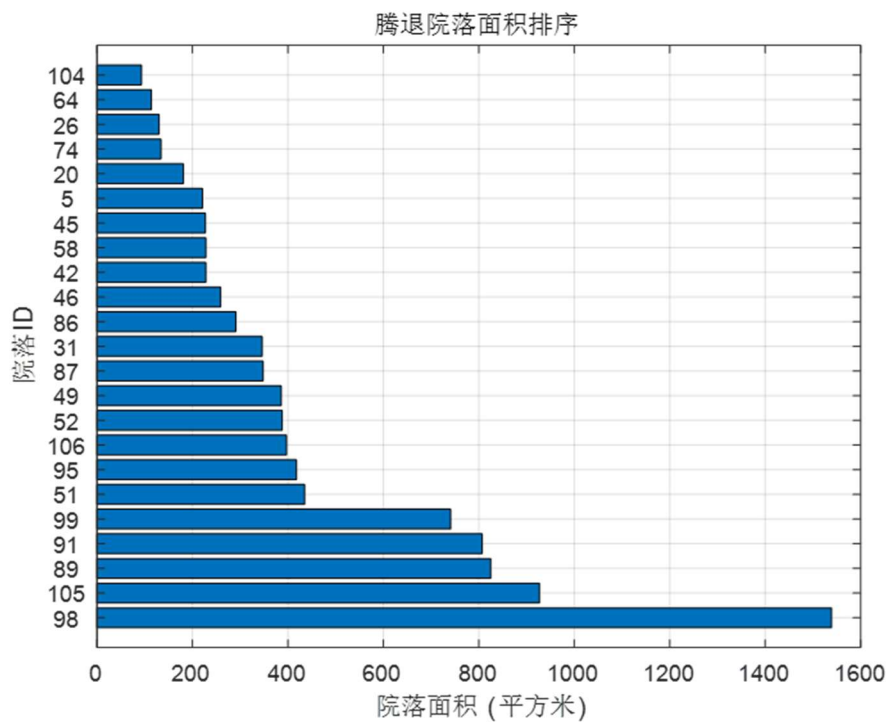


图 4.7 腾退院落面积排序柱状图

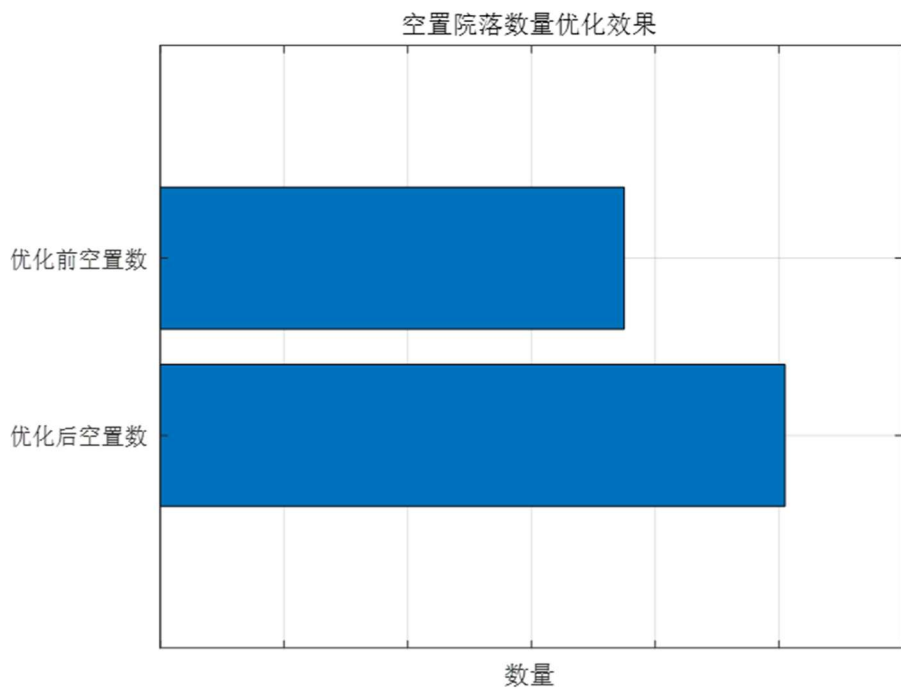


图 4.8 搬迁前后空置院落优化示意图

图 4.9 和图 4.10 分别展示了不同时间对于各项投入成本和租金收入的比例示意，随着时间即出租天数由 365 天增加到 3650 天这个过程，面积损失成本的比例扩大，其他投入成本的比例缩小，而租金方面的比例未发生变化。产生以上结果的原因在于：对于各项投入成本而言，沟通成本、补偿费用（修缮成本）、时间成本的计算公式中未与时间建立联系，即这三项成本在确定的搬迁方案条件下为固定值，而面积损失成本计算公

式中具备时间因素，面积损失成本与时间呈正比例关系，故时间越长其面积损失成本值就越大，进而相比起固定的其他三项成本而言，面积损失成本比例变大，而其他投入成本比例变小；对于租金收入而言，整院租金、毗邻奖励和零散地块都与时间呈正比例关系，故当时间发生变化时，三者的比例并不发生波动。即对投入成本和租金作关于出租/开发的时间灵敏度分析，可以得到类似以下图的规律如上文所述。

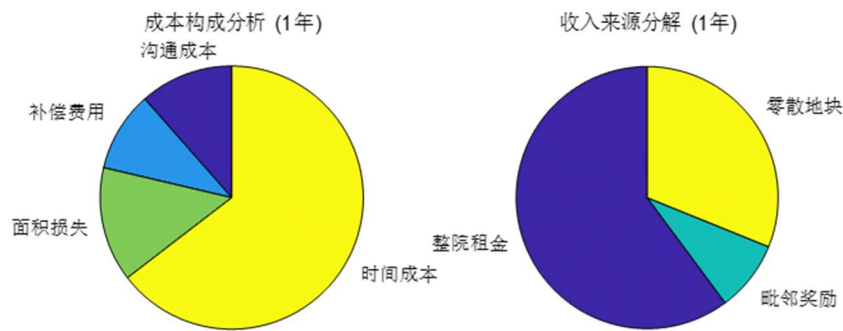


图 4.9 搬迁一年投入成本/租金收入来源饼状图

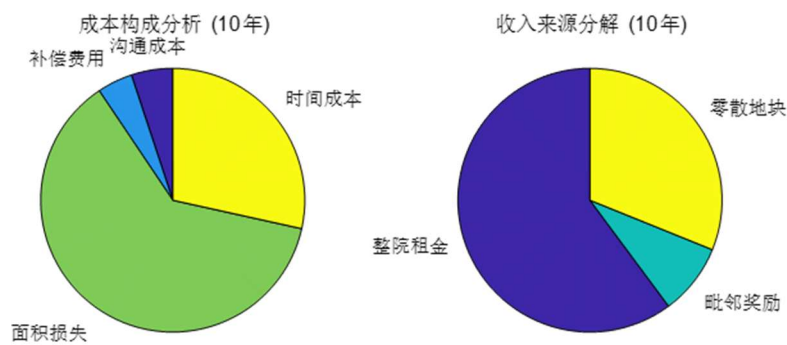


图 4.10 搬迁十年投入成本/租金收入来源饼状图

五、问题三的模型建立与求解

5.1 问题分析

在问题二的基础上，本文已求得一组可行解即住户搬迁方案，腾出了一批整院用于出租/开发。为衡量搬迁的经济合理性，在不考虑保底规划成本约束的情况下，开发商希望以十年为期限，评估搬迁方案的投资回报率，并确定是否存在一个“性价增益拐点”。若性价比高于某阈值（题设为 $m \geq 20$ ），则该搬迁是值得投资的。在 $m \geq 20$ 的条件下，若存在搬迁拐点，需明确拐点对应的搬迁方案；若不存在，需确定 m 的最低值以使得拐点存在。

问题三的核心在于识别“投资回报率开始递减”的关键拐点，即继续搬迁所带来的收益增长低于投入增加所对应的点。根据题意，定义 m 为“十年租金净增益除以实际搬迁投入”，并以 $m \geq 20$ 作为开发商预期的经济回报下限。

若在 $m \geq 20$ 时存在拐点，即在连续搬迁的过程中，在第 $n+1$ 住户搬迁后是否满足 $m_n > m_{n+1}$ ，且 $m_n \geq 20$ ，此时认为该点为“性价增益的搬迁拐点”，该拐点对应的搬迁决

策方案为最优决策方案。

若在 $m \geq 20$ 时不存在拐点，这时需要考虑两种情形，一是所有搬迁规模对应的 m 在小于 20 时均不满足开发商的收益预期。对于这种情况，可以利用 m 的 MILP 数学模型，求解 m 的数值大小，并用来反推 m 的最低阈值，判断 m 降低到至少多少能存在拐点。二是 m 始终单调递增，搬迁规模越大，性价比越高。对于这种情况，可直接输出最大搬迁规模和 m 的递增情况。

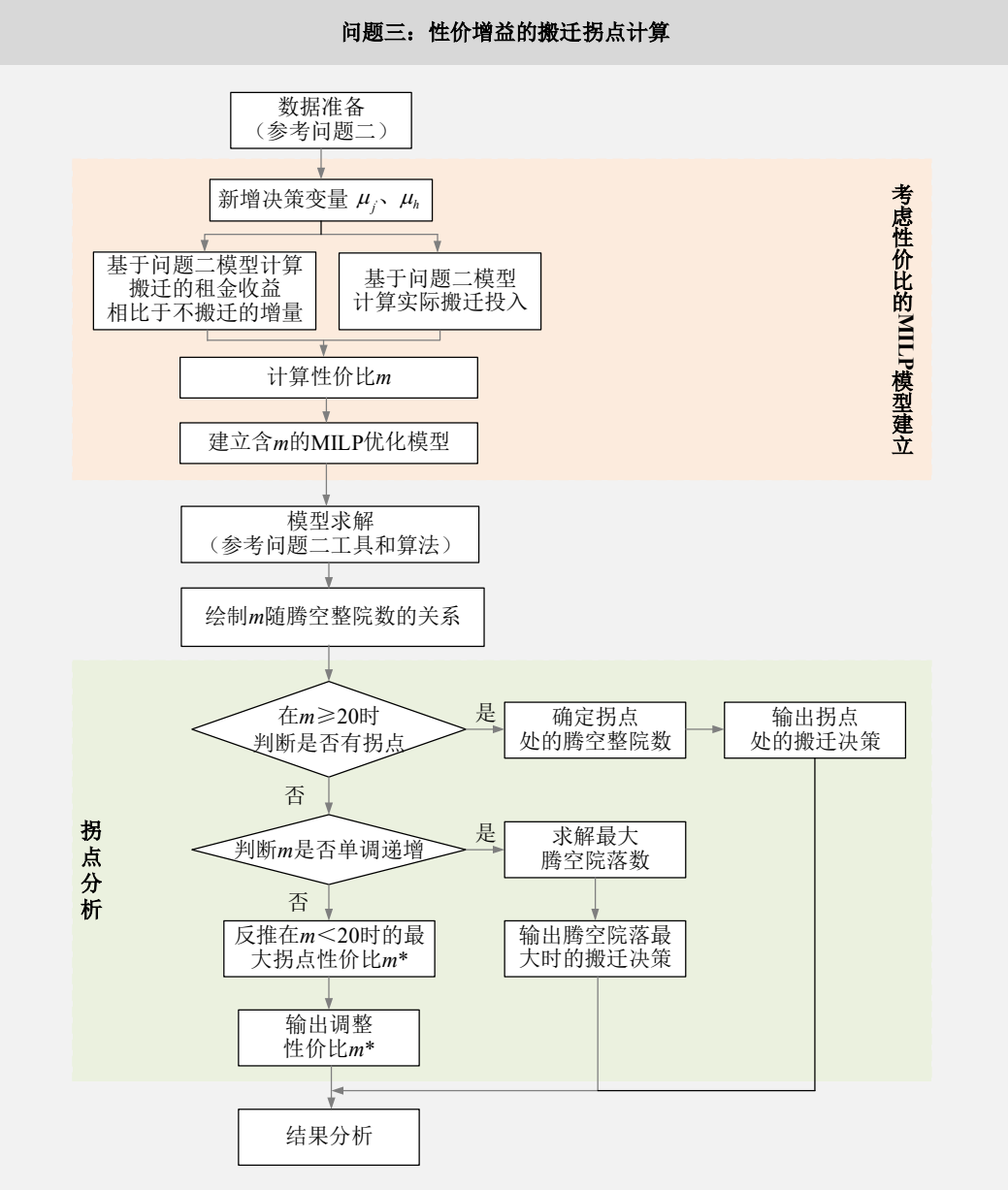


图 5.1 问题三性价比增益的搬迁拐点计算流程

5.2 建立性价比增益的搬迁性价比拐点规划模型

首先，对不搬迁的十年租金收益进行数据预处理。由于初始状态的院落情况是已定的，根据附件数据以及院落毗邻矩阵表格，租金收益可以由附件中的代码文件“不搬迁十年租金计算.py”直接计算，计算结果为。

表 5.1 不搬迁的初始数据

新增符号	定义描述
每日租金收益（不搬迁）	762701

十年总收益（不搬迁）	2783858650
整院院落数	41
非毗邻整院数	18
分散空置地块数	141

以问题二建立的模型为基础，建立问题三的模型，本问题的目标是计算搬迁的性价比演化过程，寻找收益递减的“搬迁拐点”，在建模时，通过逐步模拟搬迁进程的方式，从高性价比的搬迁方案开始，依次加入搬迁对象，计算特定步长的新增收益和成本，并绘制出性价比随搬迁规模变化的曲线，从而识别可能存在的拐点区间。若不存在拐点，则推导在不同 m 阈值下能否建立经济条件，为实际政策的制定提供调节空间。

在取消成本上限的情况下，考察十年增量租金与实际搬迁投入之比 m ，并在求得 m 达到最大的条件下，生成相应的住户搬迁情况，

$$m = \frac{R_1 - R_0}{\sum_{h \in H} (C_h^m + C_h^a + C_h^r + C_h^t)} \quad (5-1)$$

其中， $R_1 - R_0$ 表示搬迁十年的租金收益相比于不搬迁的增量， $\sum_{h \in H} (C_h^m + C_h^a + C_h^r + C_h^t)$ 表示实际搬迁投入。

由于题设以搬迁十年为条件，故将出租天数 T 取值为 3650（一年以 365 天计），即 $T = 3650$ 将 T 代入到问题二的公式中从而得到不搬迁的十年租金收益、搬迁后的十年租金收益以及搬迁后十年的面积损失成本，分别计算如下公式

$$R_0 = 3650 \cdot \left(\sum_{i \in I} 30 \cdot A_i \cdot z_i^0 + \sum_{i,k \in I} 30 \cdot (A_i + A_k) \cdot 0.5 \cdot 0.2 w_{ik}^0 + \sum_{i \in I} \sum_{j' \in J'} (1 - z_i^0) \cdot v_{j'}^0 \cdot a_{j'} \right) \quad (5-2)$$

$$R_1 = 3650 \cdot \left(\sum_{i \in I} 30 \cdot A_i \cdot z_i + \sum_{i,k \in I} 30 \cdot (A_i + A_k) \cdot 0.5 \cdot 0.2 w_{ik} + \sum_{i \in I} \sum_{j \in J} (1 - z_i) \cdot v_j \cdot a_j \cdot u_j \right) \quad (5-3)$$

$$C_h^a = 3650 \cdot \sum_{j' \in J'} v_{j'} \cdot (a_{j'} - a_h) \cdot y_{j'}^h, \forall h \in H \quad (5-4)$$

其中，公式（5-2）的计算包括初始整院院落租金收益、初始毗邻整院院落租金收益以及初始分散空置地块租金收益，为固定值。由代码计算得到， $R_0 = 2783858650$ 。公式（5-3）的计算包括整院院落租金收益、毗邻整院院落租金收益及分散空置地块租金收益。

所有初始未入住的地块 j' 在搬迁决策实施后是否为分散空置地块可以由 $\sum_{h \in H} y_{j'}^h$ 判断，若 $\sum_{h \in H} y_{j'}^h = 1$ 则表示原属于 J' 即空置地块里的地块 j' 搬入了住户，此时该地块 j' 的状态转变为不空置，即 $u_{j'} = 0$ ；若 $\sum_{h \in H} y_{j'}^h = 0$ 则表示原属于 J' 即空置地块里的地块 j' 没有搬入住户，此时该地块 j' 的状态仍为空置。经分析可知， $u_{j'}$ 的表达式如下：

$$u_{j'} = 1 - \sum_{h \in H} y_{j'}^h, \forall j' \in J' \quad (5-5)$$

同理，对于所有初始入住地块 h ，其在搬迁决策实施后是否为分散空置地块可以由 $\sum_{j' \in J'} y_{j'}^h$ 判断，若 $\sum_{j' \in J'} y_{j'}^h = 1$ 则表示原属于 H 即原住户地块里的地块 h 搬出了住户，此时该地块 h 的状态转变为空置即 $u_h = 0$ ；若 $\sum_{j' \in J'} y_{j'}^h = 0$ 则表示原属于 H 即空置地块里的地块 h 没有搬出住户，此时该地块 h 的状态仍为不空置。经分析可知， u_j 的表达式如下：

$$u_h = \sum_{j' \in J'} y_{j'}^h, \forall h \in H \quad (5-6)$$

$$y_{j'}^h, z_i, w_{ik}, \eta_{j'}^h, u_j \in \{0, 1\}, \forall i, k \in I, h \in H, j \in J, j' \in J' \quad (5-7)$$

由于问题三题设为不考虑保底规划成本约束，故选择启用备用金为最大经济条件即

公式 (4-16)。其余公式均与问题二中约束条件表述相同，约束条件对应公式 (4-3)、公式 (4-5) - (4-7)、公式 (4-9) - (4-14)、公式 (4-16) - (4-23) 以及公式 (5-1) - (5-7)。

综上，问题三的数学模型为公式 (4-3)、公式 (4-5) - (4-7)、公式 (4-9) - (4-14)、公式 (4-16) - (4-23) 以及公式 (5-1) - (5-7)。

5.3 模型求解

问题三的关键在于性价比 m 的计算与应用。对 m 进行详细分析可知， m 的值随着居民的搬迁而变化，但由于居民的搬迁与地块相关联，计算量过大导致模型难以求解，故考虑将 m 与其他指标关联。

对整个问题进行分析， m 的取值首先受到净收益的影响，并且在一定范围内与净收益呈正比（见图 5.2），而当 m 的值不断增大，可能会出现收益增加但 m 减小的情况，也就是所谓的“拐点”。在求解收益的过程中，有两个很重要的指标：空置的院落数量与毗邻的院落数量。为了使得这两个指标尽可能大，居民需要搬迁以腾空原所属院落，也就是说居民的搬迁目的就是为了进行腾空院落的操作。因此，倒腾空置院落数与性价比具有很强的关联性。若不考虑性价比，根据问题 2 的求解结果可知倒腾空置院落数最大为 23，相比于搬迁的居民数来说运算量大大减小。建立倒腾空置院落数与性价比 m 的关系，倒腾空置院落数的增大代表着居民搬迁正在进行。因此将求 m 拐点的问题转化为求倒腾空置院落数与性价比 m 曲线的拐点问题。若在 $m \geq 20$ 的基础上存在拐点，则输出当前倒腾空置院落数下的居民搬迁方案与总收益等结果，若不存在拐点，判断根据曲线是否单增，若单增，则判断 m 的最大值 m^* 是否在 $m \geq 20$ 上，若是则输出当前的搬迁方案，若不在则记录 m^* 的值，表示当前所有方案都达不到性价比要求。若函数不单增，说明在 $m \geq 20$ 无拐点，但在 $m < 20$ 的区间 m 是非单增且有最大值的。那么 m 的最大值 m^* 即为潜在的拐点，当性价比的认为界限变为 $m \geq a, a < m^*$ 时即可出现拐点。其中，在计算 0 个腾空院落数的 m 的值时，为保证合理性将其设置为 0，即忽略搬迁了几个居民但未腾空出院落的情况。

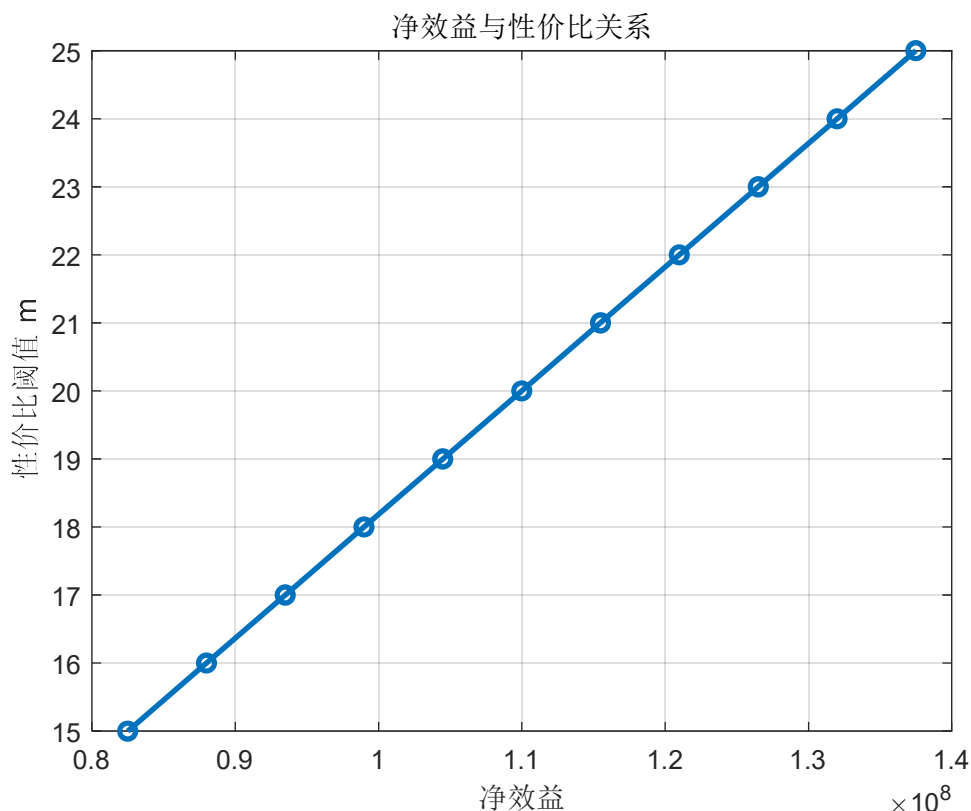


图 5.2 净收益与性价比关系图

5.4 仿真分析（或者结果分析）

根据问题三上述求解思路，设计代码并在 matlab 中运行后得出的 m 与倒腾空置院落数的曲线如下图所示：

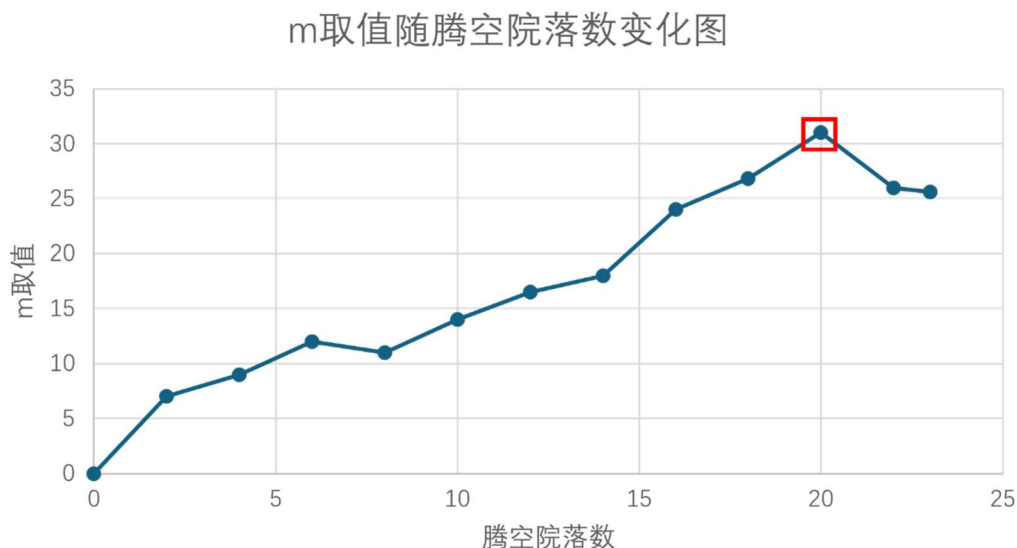


图 5.3 m 取值随腾空院落数变化图

可以发现，算法识别的图像拐点位于腾空院落数=20 处，即在院落数小于 20 时， m 的取值随着腾空院落的增加而增加，在腾空院落数范围为 5-10 时出现局部最优解但马上回升，可能是由于搬迁某居民时付出的成本过高导致整体性价比变低。当腾空院落数>20 时开始下降，因此，提取腾空院落数为 20 时的搬迁方案如下：

表 5.2 搬迁的住户 OD 表

搬迁前 ID	搬迁后 ID	搬迁前 ID	搬迁后 ID
14	357	215	115
91	19	224	83
92	2	225	467
93	281	235	257
110	4	296	170
111	119	297	136
138	173	327	214
139	247	380	51
140	277	381	123
141	183	384	213
142	229	385	399
179	290	393	422
180	300	409	78
181	58	424	70
182	341	433	387
200	426	468	273
203	209	469	124

204	130	
-----	-----	--

表 5.3 搬迁地块-院落数据结果表

数据名称	数据结果
总搬迁住户数	35 个
完全腾退院落数量	20 个
腾退院落的总面积	8415.00 m ²
完全腾退院落 ID	[5,20,26,31,42,45,46,49,51,52,64,74,86,87,89,91,95,98,99, 106]
空置院落总面积	29526 m ²

表 5.4 搬迁投入成本数据结果表（十年）

数据名称	数据结果
沟通成本	1050000 元
修缮成本	921000 元
面积损失成本	12006200 元
时间损失成本	6064360 元
投入总成本	20049560 元

表 5-4 搬迁租金收益数据结果表（十年）

数据名称	数据结果
整院租金收益	2800681500 元
毗邻院落额外租金收益	420545700 元
分散空置地块租金收益	177192900 元
搬迁十年后总收益	3398420100 元

可以发现相较于问题 2，虽然在腾空院落为 20 个时提前输出了结果导致总收益有所减小，但总成本尤其是面积损失成本有了较大的减少，因此投资的性价比更高。对搬迁方案进行分析可见腾空的院落被包含于问题 2 的腾空院落集合中，并且搬迁方案对非常类似，说明腾空院落数与 m 的关联性与地块相类似。对求解过程进行分析，整个问题没有出现无拐点的情况与 m 单调递增的情况。

六、问题四的模型建立与求解

6.1 问题分析

问题四需要明确输入参数和计算步骤，并设计一个智能老城区平移置换决策的软件框架。基于前面的建模成果，该系统具备灵活的参数输入界面，允许用户通过角色匹配进行不同的参数设置，并能自动调用问题二与问题三的核心模型，完成最优搬迁方案的决策和指标的计算，软件的功能模块分为：地块数据预处理、用户角色判断、最优搬迁决策规划、量化指标计算和交互式结果展示，实现了对老旧城区更新搬迁工作的自动化和可视化。以 Matlab 为程序基础，结合 Figma 构建前端界面，直观展示了搬迁决策的前端软件操作和后端计算步骤，以实现住户搬迁地块综合评价、不考虑性价比（问题二）和考虑性价比（问题三）的开发商搬迁决策三重功能。

6.2 智能决策软件需求分析

软件的功能模块分为：地块数据预处理、人工参数设置、最优搬迁方案决策、租金

收益计算和交互式结果展示。

(1) 地块数据预处理

根据需要进行平移置换的街区，系统应提前设置好街区图示、地块详细信息和其他系统参数。

(2) 人工参数设置

从住户的角度出发，期望设置搬迁补偿模型的各项影响因素。从开发商的角度出发，期望设置成本、利润、性价比等参数。

(3) 最优搬迁路径规划

根据用户需求，利用问题二和问题三建立的优化模型来计算最佳搬迁路径，从而生成智能搬迁决策。

(4) 租金收益和规划结果展示

计算最佳搬迁方案对应的租金收益，规划结果通过表格或者图片的形式展示。

(5) 其他

规划软件在满足核心功能的同时，还要兼顾用户体验，设计友好的用户界面。用户界面主要完成参数信息输入、结果可视化的功能。

6.3 智能决策软件总体设计

根据软件需求分析，对智能决策软件进行总体设计，软件整体框架由前端用户界面与后台决策程序两部分组成，如图 6.1 所示。

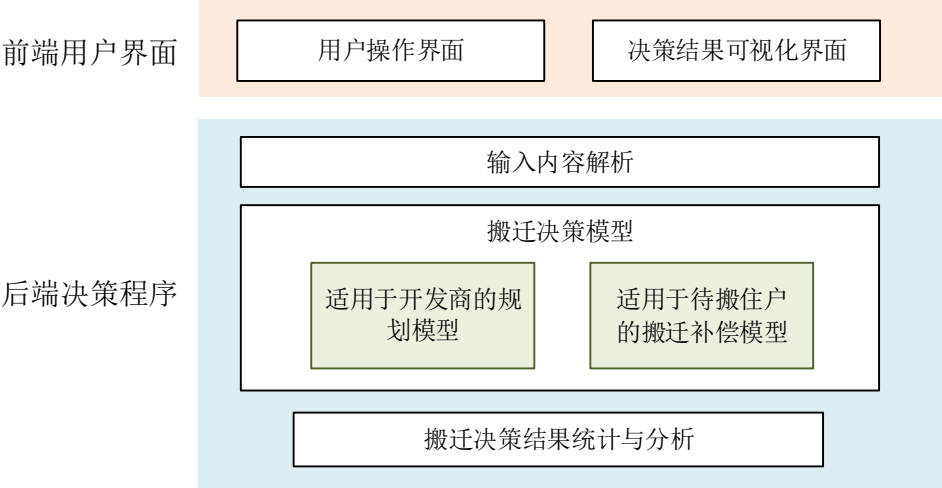


图 6.1 搬迁智能决策软件总体框架设计

从结构上看，前端用户界面集成用户操作界面与决策结果可视化界面，主要负责软件与用户的交互；后台决策程序是软件的核心，承担了通过智能决策模型和算法规划搬迁路径和计算租金收益等工作。后台决策程序可以进一步划分为三个主要模块，其中，输入参数解析模块提供前端用户界面与后台决策程序的交互接口；搬迁决策算法模块集成前述问题的模型和算法，结合基础数据与输入信息，实现具体的搬迁决策工作；搬迁决策结果统计与分析模块通过对决策算法在运行过程中的各项数据进行记录与整理，生成不同地块间的搬移路线，并提供租金收益等指标结果。

6.4 智能决策软件设计

6.4.1 前端用户界面功能结构

前端用户界面的功能结构如图 6.2 所示，页面视图层包括基础数据、角色匹配、参数设置、决策结果、历史决策与帮助共 6 个子页面。数据平台层主要实现基础数据和设置参数的存储与维护功能。同时，还负责决策结果的存储工作。功能平台层提供了输入

信息处理与决策结果解析模块，输入信息处理模块负责整理各类输入参数，调用系统文件输出接口，并将各类输入参数输出为描述文件。决策结果解析模块通过调用系统文件输入接口，读取决策结果文件，并写入数据平台。

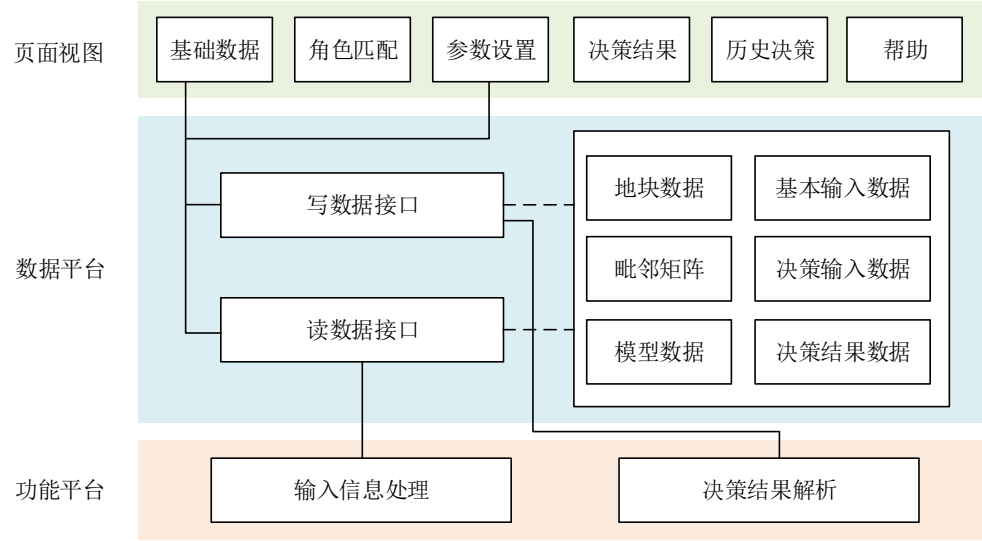


图 6.2 前端用户界面功能结构

6.4.2 基础数据和输入参数

主要软件基本数据和输入参数的介绍和格式如下：

- (1) 系统基础数据
 - ① 地块信息表 (excel)：包含地块 ID、院落 ID、地块面积、地块的朝向、地块内是否有居民居住、地块所属院落等。
 - ② 院落关系图或表：给出院落之间是否毗邻的关系图，并转化为毗邻矩阵表。
 - ③ 面积补偿上限：输入补偿面积与原面积的比例不能超过多少（原问题为 0.3）。
 - ④ 采光排序：根据特定需求输出四个朝向的采光价值排序（如原问题为南厢=北厢>东厢>西厢）。
 - ⑤ 满意度阈值。
 - ⑤ 其他数据，如院落之间的距离表、各院落与街道之间的距离表等。
- (2) 住户输入参数
 - ① 居民对面积补偿的敏感权重 α ：输入居民对搬迁时给出面积补偿的敏感权重， $\alpha \in [0,1]$ 。
 - ② 居民对采光补偿的敏感权重 β ：输入居民对搬迁时给出采光补偿的敏感权重， $\beta \in [0,1]$ 。
 - ③ 居民对修缮补偿的敏感权重 γ ：输入居民对搬迁时给出修缮补偿的敏感权重， $\gamma \in [0,1]$ 。
 - ④ 居民对空间句法指标的敏感权重 μ ：输入居民对搬入地块的空间句法指标的敏感权重， $\mu \in [0,1]$ 。
 - ⑤ 居民对搬迁前后距街道距离的敏感权重 δ ：输入居民对搬迁前后距街道距离的敏感权重 $\delta \in [0,1]$ 。
 - ⑥ 居民对搬迁前后院落距离的敏感权重 σ ：输入居民对搬迁前后院落距离的敏感权重 $\sigma \in [0,1]$ 。

(3) 开发商输入参数

- ① 沟通成本：输入为促使居民搬迁的沟通成本（原问题为 3 万元）。
- ② 最大修缮补偿：输入开发商愿意提供的每户最大修缮补偿（原问题为 20 万元）。

- ③ 搬迁时间空挡：输入因搬迁带来的地块时间空档（原问题为 4 个月）。
- ④ 投资收益期：输入要计算总收入的投资收益期 T （原问题为 10 年）。
- ⑤ 整院出租单价：输入空置整院的出租单价收益（原问题为 30/平方米/天）。
- ⑥ 毗邻院落价值系数：输入为毗邻院落产生的额外价值系数（原问题为 1.2）。
- ⑦ 零散出租单价：输入为零散院落时各朝向地块的出租单价（原问题南=北=15/平方米/天，东=西=8/平方米/day）。
- ⑧ 性价比期望下限 m_0 ：（原问题为 20，根据是否考虑性价比来决定是否输入）

6.4.3 前端用户界面设计

图 6.3 给出了采用 Figma 软件设计的用户界面，当用户登录系统后，在导航栏可以看到基础数据、角色匹配、参数输入、历史决策和帮助六个主要模块。

基础数据模块主要展示所在老旧街区的地图、地块详细信息以及其他参数设置，对应于系统基本数据，这些由当地政府或规划局预先设置完成，不能修改。角色匹配模块主要是为了确认用户是住户还是开发商。如果是住户，则在参数输入模块需要设置上述列出的住户输入参数，如果是开发商，则在参数输入模块需要设置上述列出的开发商输入参数，并给出是否考虑性价比的判断。输入参数完成并运行后，在决策结果模块可以看到生成的结果。决策结果模块能够为住户提供是否建议搬迁的提示，若建议搬迁，则给出满足搬迁补偿条件的所有地块信息。至于开发商，能够为其提供搬迁建议表、空置院落表、毗邻空置表等决策信息。历史决策模块能够存储用户之前的决策结果。帮助模块为用户熟悉和理解软件提供帮助。由于时间原因，完整的软件操作步骤需要后续进一步完善。



图 6.3 Figma 设计前端界面

6.4.4 后台决策程序功能结构

后台调度程序通过搬迁决策模型和算法，完成老旧街区平移置换决策功能，划分为

输入参数解析、搬迁决策算法、决策结果统计与分析三个顶层模块，其基本功能结构如图 6.4 所示：

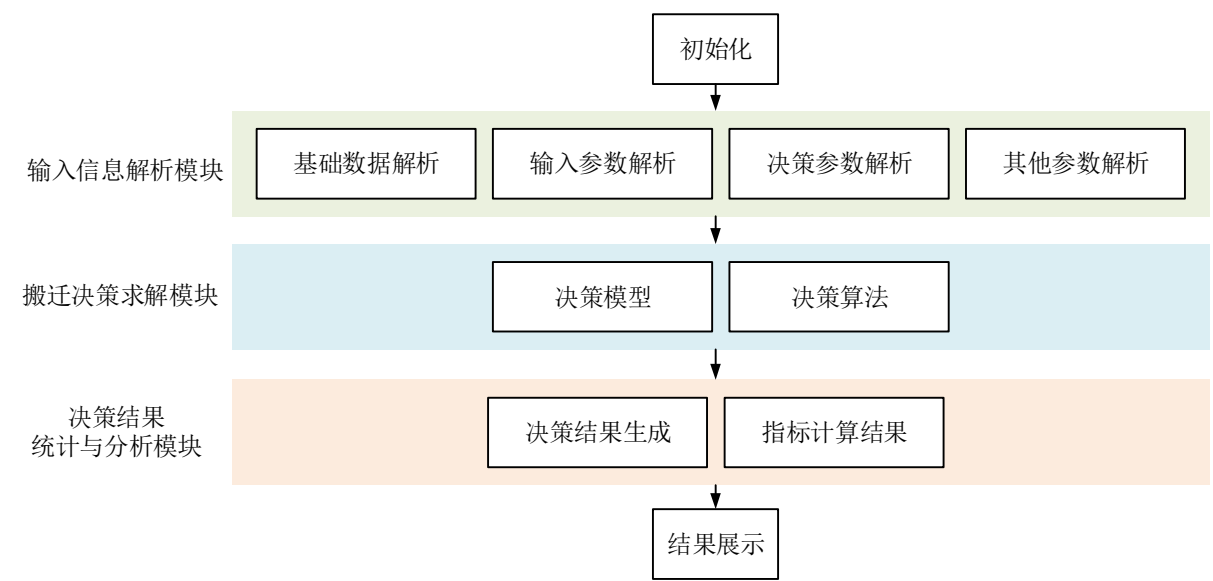


图 6.4 后台决策程序功能结构

（1）输入信息解析

输入参数解析模块是智能决策软件的入口，它负责将前端用户预定义参数转化成后台搬迁决策程序可用的数据信息，以供其他模块完成初始化配置工作。该模块通过系统读取文件接口，读取相应的描述文件，将读取到的数据进行处理后，分别用于实例化算法参数类、决策参数类以及决策目标集合。

（2）搬迁决策求解

基于搬迁决策数学模型和优化算法进行搬迁路径规划，按照后台决策程序整体架构实现智能决策算法各功能模块。

（3）决策结果统计与分析

决策结果统计与分析模块需要记录决策结果，生成老旧街区中各地块的搬迁决策表，并对搬迁地块数、腾空整院数量、腾空面积、租金收益等指标进行统计和分析。

6.4.5 后台决策程序计算步骤

后台决策程序的计算步骤如图 6.5 所示。首先，根据基础数据和输入参数初始化后台程序，然后解析用户在前端角色匹配模块的输入，从而做出不同决策。如果用户角色是待搬住户，则需要根据住户输入的权重参数初始化问题一的搬迁补偿模型，然后分别计算每个 h 地块迁入所有空置地块的满意度。判断最大满意度是否大于满意度阈值，如果是，则继续判断其他所有满足条件的地块，提示用户建议搬迁，并提供可供迁入的地块信息。

如果用户角色是开发商，则需要判断该开放商在前端是否选择了决策要考虑性价比。若不考虑性价比，则根据开发商的输入参数，如成本、单价等初始化问题二模型，并调用 Yalmip+Gurobi 求解，最终输出搬迁决策方案和指标计算结果。考虑性价比时的计算比较复杂。首先成本、单价、性价比下限等初始化问题三模型，采用与问题二相同的求解方法，并计算 m 随搬迁规模变化的曲线。在分析 m 曲线的基础上，进一步判断其是否有拐点，如果有拐点，输出拐点处对应的搬迁决策方案和指标计算结果。若无拐点，则继续判断 m 曲线是否单调递增。如果不是单调递增，则性价比始终低于开发商预期性价比，此时可根据问题三模型反推 m 的最大拐点性价比，并输出调整后的性价比，提示用户变更。若 m 曲线既无拐点也不单调递增，则也可以分为两种情况，一是达到最大腾

空整院数量后，性价比仍然不能达到设定性价比下限，此时提示用户未生成满足性价比条件的搬迁决策方案，请重新输入性价比。二是最大性价比大于设定性价比下限，则输出最大腾空整院最大时的搬迁决策方案和指标计算结果。最后，将决策结果通过地块信息表、腾退院落分布热力图、成本收益曲线及性价比演化三维曲面等形式展现。

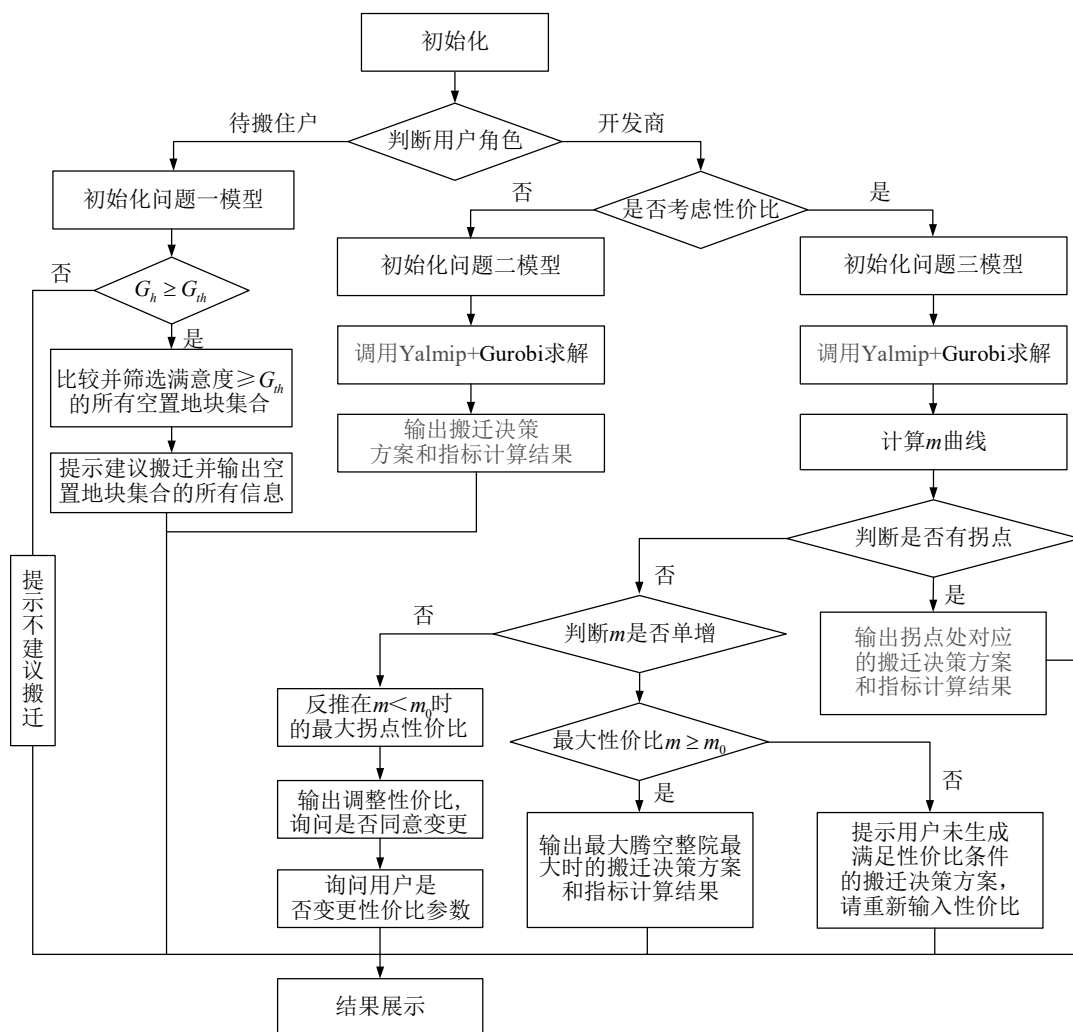


图 6.5 后台决策程序计算步骤

6.4.5 后台决策程序实现

见附录四。

七、模型评价与推广

7.1 模型的优缺点

(1) 居民搬迁意愿评价模型

优点：①通过六项影响因素综合评估居民的搬迁意愿，尤其是除题设条件外，引入搬迁距离、搬迁前后院落距街道的距离与空间句法综合指标以实现更全面反应居民需求的评价方法。

②对每项指标进行归一化并匹配相应的权重，居民可以根据自己对某项评价指标的敏感程度动态调整各项权重值以得到真实的搬迁意愿。

③根据约束输出居民的可搬入地块候选集，增强了搬迁决策的科学性。

缺点：指标的建立可以进一步丰富，同时判断是否搬迁的阈值依赖于政府或管理者

直接给定。

(2) 整院数量最大化与毗邻效益协同优化模型

优点：①通过严谨的数学公式推导建立了根据院落与地块信息计算整院数量、面积协同毗邻效益的优化模型。

②模型通过目标函数整合经济收益与成本控制，逻辑清晰，优先级分明，目标函数中明确区分整院收益、毗邻额外收益和搬迁成本，便于量化评估。

③模型充分考虑了现实中的搬迁限制条件，约束贴合实际场景。

缺点：①模型涉及多个二元变量，约束条件较为细化导致变量规模较大且模型的维数较高，需依赖高性能计算资源。

②模型在约束中存在部分约束线性化不足，一定程度上增加了算法计算时间。

③在考虑经济效益时，假设租金、修缮成本、采光赋值等参数稳定不变，忽略市场波动、政策调整等动态因素，仅考虑一次性搬迁的调整需求。

(3) 性价增益的搬迁性价比拐点规划模型

优点：①模型针对“是否存在性价比拐点”这一实际问题，通过定义性价比指标，将复杂的经济性评估转化为可量化的阈值判断，逻辑清晰且易于决策者理解。

②在计算性价比时，复用了问题二模型的整院约束、毗邻收益计算等关键模块，减少重复建模成本，考虑了不同情况下的地块租金收益，从理论上增加了模型的可靠程度。

③模型的推进严格基于所给数据与公式进行，准确性高，可应用性强。

缺点：由于所给条件有限，计算各搬迁方案性价比过程中存在一定假设，如计算性价增益效应时，按照最高成本即备用金为成本约束，在一定程度上会影响模型精度与可视化的性价比曲线。

(4) 老城区平移置换决策软件设计模型

优点：①充分考虑到不同人群对软件的不同要求，将软件设计成居民与开发者两端。

②详细给出了软件运行的计算过程与逻辑，很好的满足题目给出的各种要求。

③给出了示例的软件界面（手绘），为软件制作提供了范例。

缺点：由于时间原因，具体的软件制作代码没有完成。

7.2 模型推广

(1) 本研究所建立的搬迁决策模型核心是在有限资源下的空间配置优化和多目标权衡决策模型，不仅适用于当前老城街区的平移置换搬迁规划，还可推广至全国范围内的老旧城区更新项目，为城市更新提供科学决策支持，并且，除了可以推广到其他老旧城区更新搬迁的决策中去，还能广泛应用于其他非搬迁类场景的其他城市规划中。例如，医院床位资源调配、城市停车位的布局优化、公共交通工具的调度等。

(2) 本研究提出的“平移置换”智能规划模型具备高度可扩展性，可推广至全国不同城市的老旧街区更新项目，引入标准化数据接口，兼容各城市的 GIS 地图、产权数据和居民信息，实现“一城一策”精准规划。在历史保护区，可增设“文物建筑保护约束”，禁止搬迁特定院落；针对数据不完整的老旧街区，可以采用插值法或机器学习预测缺失值；在商业密集区，接入人口密度、商业热点分布等开放数据，可建立优化租金收益模型。通过动态调整目标函数与约束条件，模型能够灵活响应不同城市的更新需求，为全国城镇化提质提供统一技术框架。

参考文献

[1] 翁洁岚. 新时代城市房屋拆迁补偿问题研究 [D]. 福建师范大学, 2020. DOI:10.27019/d.cnki.gfjsu.2020.002387.

[2] 赵冲, 甘国乾臻, 黄宇东. 基于空间句法的福清传统合院式民居空间形态研究[J]. 室内设计与装修, 2025, (04): 121-123.

[3] 区金全. 基于演化博弈的广州旧村庄改造拆迁补偿研究 [D]. 华南理工大学, 2023. DOI:10.27151/d.cnki.ghnlu.2023.001672.

[4] 郑耀贤. 深圳市城中村改造项目居民满意度评价及影响因素研究 [D]. 深圳大学, 2023. DOI:10.27321/d.cnki.gszdu.2023.002723.

[5] 王天龙. 互联互通模式下的城轨车辆调度优化模型研究 [D]. 重庆交通大学, 2024. DOI:10.27671/d.cnki.gcjtc.2024.000538.

[6] 申东, 谭昕, 王佳豪, 等. 一种基于 Gurobi 的雷达通信一体化波形优化设计方法[J]. 空天预警研究学报, 2025, 39(01): 1-6+23.

[7] 周隽, 王天淇. 基于分支定界法的机场终端区单一进离场程序设计优化[J]. 计算机科学, 2020, 47(S1): 552-555+571.

附录 1：问题一算法程序

程序编号	1	程序说明	搬迁补偿模型建立	编程软件	Matlab
%% 辅助函数: 构建约束条件					
function [A, b, Aeq, beq, lb, ub] = buildConstraints(num_residents, num_vacant_plots, num_courtyards, adjacency_matrix, ...					
eligible_map, courtyard_occupied_plots, cost_map, obj_f, m_threshold, occupied_plots)					
num_residents_vars = num_residents * num_vacant_plots;					
num_courtyard_vars = num_courtyards;					
adjacent_pairs_count = sum(sum(adjacency_matrix)) / 2;					
total_vars = num_residents_vars + num_courtyard_vars + adjacent_pairs_count;					
max_constraints = num_residents_vars + num_residents + num_vacant_plots + ...					
sum(cellfun(@length, courtyard_occupied_plots)) + ...					
2 * adjacent_pairs_count + 1;					
A = zeros(max_constraints, total_vars);					
b = zeros(max_constraints, 1);					
constraint_row = 1;					
% 1. 可行性约束: x_ij <= e_ij					
for i = 1:num_residents					
for j = 1:num_vacant_plots					
var_idx = (i-1) * num_vacant_plots + j;					
A(constraint_row, var_idx) = 1;					
b(constraint_row) = eligible_map(i, j);					
constraint_row = constraint_row + 1;					
end					
end					

```

% 2. 居民安置约束: 每个居民最多搬到一个地块
for i = 1:num_residents
    for j = 1:num_vacant_plots
        var_idx = (i-1) * num_vacant_plots + j;
        A(constraint_row, var_idx) = 1;
    end
    b(constraint_row) = 1;
    constraint_row = constraint_row + 1;
end

% 3. 地块占用约束: 每个空置地块最多被一个居民占用
for j = 1:num_vacant_plots
    for i = 1:num_residents
        var_idx = (i-1) * num_vacant_plots + j;
        A(constraint_row, var_idx) = 1;
    end
    b(constraint_row) = 1;
    constraint_row = constraint_row + 1;
end

% 4. 院落腾退约束
for k = 1:num_courtyards
    residents_in_courtyard = courtyard_occupied_plots{k};
    if ~isempty(residents_in_courtyard)
        courtyard_var_idx = num_residents_vars + k;

        % 对院落中的每个居民
        for res_idx = 1:length(residents_in_courtyard)
            i = find(occupied_plots == residents_in_courtyard(res_idx));
            if ~isempty(i)
                %  $y_k \leq \sum_j x_{ij}$ 
                temp_row = zeros(1, total_vars);
                temp_row(courtyard_var_idx) = 1;

                for j = 1:num_vacant_plots
                    var_idx = (i-1) * num_vacant_plots + j;
                    temp_row(var_idx) = -1;
                end

                A(constraint_row, :) = temp_row;
                b(constraint_row) = 0;
                constraint_row = constraint_row + 1;
            end
        end
    end
end
end

```

```

end

% 5. 相邻院落对约束
adjacent_offset = num_residents_vars + num_courtyard_vars;
adjacent_idx = 1;

for i = 1:num_courtyards
    for j = i+1:num_courtyards
        if adjacency_matrix(i,j) == 1
            % z_ij <= y_i
            A(constraint_row, adjacent_offset + adjacent_idx) = 1;
            A(constraint_row, num_residents_vars + i) = -1;
            b(constraint_row) = 0;
            constraint_row = constraint_row + 1;

            % z_ij <= y_j
            A(constraint_row, adjacent_offset + adjacent_idx) = 1;
            A(constraint_row, num_residents_vars + j) = -1;
            b(constraint_row) = 0;
            constraint_row = constraint_row + 1;

            adjacent_idx = adjacent_idx + 1;
        end
    end
end

% 6. 性价比约束: 十年总租金收益 >= m * 总投入

rent_row = zeros(1, total_vars);
rent_row(num_residents_vars+1:end) = obj_f(num_residents_vars+1:end);
for i = 1:num_residents
    for j = 1:num_vacant_plots
        var_idx = (i-1) * num_vacant_plots + j;
        rent_row(var_idx) = -m_threshold * cost_map(i, j);
    end
end

A(constraint_row, :) = rent_row;
b(constraint_row) = 0;
constraint_row = constraint_row + 1;

A = A(1:constraint_row-1, :);
b = b(1:constraint_row-1);
Aeq = [];
beq = [];

```

```

lb = zeros(total_vars, 1);
ub = ones(total_vars, 1);
end

%% 读取 Excel 数据
data = readtable('附件一：老城街区地块信息表.xlsx');
num_plots = height(data);
%% 使用结构体和 Map 容器替代动态变量（避免 eval）
plots = struct(); % 存储所有地块信息
H = [];           % 有住户地块 ID 集合
jj = [];          % 无住户地块 ID 集合
I = containers.Map('KeyType', 'double', 'ValueType', 'any'); % 院落集合
%% 遍历每一行数据
for n = 1:height(data)
    % 提取关键字段（显式处理数据类型）
    plot_id = data.AAAID(n);
    courtyard_id = data.BBBID(n);
    has_resident = data.DDD(n);

    % 存储地块信息到结构体
    plots(n).ID = plot_id;
    plots(n).aj = data.AAAaria(n);
    plots(n).sj = data.FFFA(n);
    plots(n).courtyard_id = courtyard_id;

    % 分离住户与非住户
    if has_resident == 1
        H(end+1) = plot_id;
    else
        jj(end+1) = plot_id;
    end

    % 构建院落集合
    if isKey(I, courtyard_id)
        I(courtyard_id) = [I(courtyard_id), plot_id];
    else
        I(courtyard_id) = plot_id;
    end
end

%% 院落面积处理（增加一致性检查）
courtyard_area_map = containers.Map('KeyType', 'double', 'ValueType', 'double');
unique_courtyards = unique(data.BBBID);
for courtyard = unique_courtyards'
    % 提取所有匹配行的院落面积

```

```

all_areas = data.BBBaria(data.BBBID == courtyard);
% 检查是否所有面积相同
if ~all(all_areas == all_areas(1))
    error('院落 ID %d 的面积不一致, 请检查数据!', courtyard);
end
courtyard_area_map(courtyard) = all_areas(1); % 存入第一个值
end
%% 验证输出
disp(['成功读取 ', num2str(height(data)), ' 条地块数据']);
disp('-----前 5 条地块信息-----');
disp(struct2table(plots(1:5)));
disp(['有住户地块数量: ', num2str(length(H))]);
disp(['院落总数: ', num2str(length(keys(I)))]);
% 显示前 3 个院落的面积
keys_I = keys(courtyard_area_map);
disp('-----院落面积匹配示例-----');
for i = 1:min(10, length(keys_I))
    courtyard_id = keys_I{i};
    disp(['院落 ', num2str(courtyard_id), ' 的面积 : ',
num2str(courtyard_area_map(courtyard_id))]);
end
function relocationRecommendation(alpha, beta, gamma, mu, delta, sigma, theta)
% 读取地块信息表（假设为 CSV 文件）
data = readtable('地块信息表.csv');

% 分割未搬地块和已搬地块
H = data(data.is_relocated == 0, :); % 未搬地块
J_prime = data(data.is_relocated == 1, :); % 已搬地块

if isempty(J_prime)
    disp('没有可搬迁的地块');
    return;
end

for h_idx = 1:height(H)
    h = H(h_idx, :);
    fprintf('\n 处理未搬地块 %d:\n', h.id);

    % 预计算  $\Delta D$  和  $L$  的最大值
    [delta_D_max, L_max] = precompute_max_values(h, J_prime);

    % 计算所有 j' 的满意度
    [W, j_ids] = calculate_satisfaction(h, J_prime, alpha, beta, gamma, mu, delta,
sigma, delta_D_max, L_max);

    % 获取最大满意度和推荐地块

```

```

[G_h, max_idx] = max(W);
candidates = find(W == G_h);
recommended_ids = j_ids(candidates);

% 输出结果
if G_h > theta
    fprintf('建议搬迁 (满意度: %.4f)\n 满足条件的地块编号: %s\n', ...
        G_h, num2str(recommended_ids));
else
    fprintf('不建议搬迁 (满意度: %.4f < 阈值 %.2f)\n', G_h, theta);
end
end
end

function [delta_D_max, L_max] = precompute_max_values(h, J_prime)
% 计算  $\Delta D$  和 L 的最大值
delta_D = zeros(height(J_prime), 1);
L = zeros(height(J_prime), 1);

for j = 1:height(J_prime)
    j_prime = J_prime(j, :);
    delta_D(j) = max(0, j_prime.D - h.D);
    L(j) = norm([j_prime.x - h.x, j_prime.y - h.y]); % 欧氏距离
end

delta_D_max = max(delta_D);
L_max = max(L);

% 处理除零情况
if delta_D_max == 0, delta_D_max = 1; end
if L_max == 0, L_max = 1; end
end

function [W, j_ids] = calculate_satisfaction(h, J_prime, alpha, beta, gamma, mu, delta,
sigma, delta_D_max, L_max)
W = zeros(height(J_prime), 1);
j_ids = J_prime.id;

for j = 1:height(J_prime)
    j_prime = J_prime(j, :);

    % 面积补偿 (1-1.3 约束)
    a_ratio = j_prime.a / h.a;
    a_ratio = max(1, min(a_ratio, 1.3));
    a_star = a_ratio / 1.3;

```

```

% 采光补偿 (0-2 约束)
delta_s = j_prime.s - h.s;
delta_s = max(0, min(delta_s, 2));
s_star = delta_s / 2;

% 修缮补偿 (精确匹配判断)
eta = isequaln(j_prime.s, h.s) && isequaln(j_prime.a, h.a);
C_star = (eta * h.a) / 237;

% 空间句法指标 (直接读取预处理值)
tau_star = j_prime.tau;

% 距离惩罚计算
delta_D = max(0, j_prime.D - h.D);
D_star = delta_D / delta_D_max;
L = norm([j_prime.x - h.x, j_prime.y - h.y]);
L_star = L / L_max;

% 综合满意度计算
W(j) = alpha * a_star + beta * s_star + ...
       gamma * C_star + mu * tau_star - ...
       delta * D_star - sigma * L_star;

end
end

```


附录 2：问题二算法程序

程序编号	2	程序说明	最大化收益混合整数 线性规划模型求解	编程软件	Matlab+ python
<pre> clear;clc result1_file = 'result1.xlsx'; data1 = readmatrix(result1_file); % 原始地块 plot_data_file = '附件 1.xlsx'; data2 = readmatrix(plot_data_file); load("data.mat"); Adj_matrix = data; % --- 2. 定义集合与参数 --- resident_ids = unique(data1(:,1)); empty_plot_ids = unique(data1(:,2)); N = length(resident_ids); M = length(empty_plot_ids); resident_mask = data2(:,6) == 1; resident_plot_data = data2(resident_mask, :); courtyard_ids = unique(resident_plot_data(:,2)); % 获 you 居民的院落 ID K = length(courtyard_ids); % 院落数量 % 建立映射: resident_id_map = containers.Map(resident_ids, 1:N); empty_plot_id_map = containers.Map(empty_plot_ids, 1:M); courtyard_id_map = containers.Map(courtyard_ids, 1:K); courtyard_residents = cell(1, K); </pre>					

```

for i = 1:size(resident_plot_data, 1)

    plot_id = resident_plot_data(i,1); % 地块 ID

    yard_id = resident_plot_data(i,2); % 院落 ID

    if resident_id_map.isKey(plot_id) && courtyard_id_map.isKey(yard_id)

        resident_idx = resident_id_map(plot_id);

        yard_idx = courtyard_id_map(yard_id);

        courtyard_residents{yard_idx} = [courtyard_residents{yard_idx}, resident_idx];

    end

end

% 计算每个院落的总面积

Ak_total = zeros(1, K);

for k = 1:K

    yard_id = courtyard_ids(k);

    yard_plots = resident_plot_data(resident_plot_data(:,2) == yard_id, :);

    Ak_total(k) = sum(yard_plots(:,3)); % 第 3 列是面积

end

% --- 权重系数 ---

w1 = 0.7; % 腾退院落数量权重

w2 = 0.05; % 腾退院落面积权重

w3 = 0.05; % 毗连腾退权重

w4 = 0.2; % 补偿成本权重

% 可选预算

B = 26000000;

% --- 3. 构建 intlinprog 输入 ---

```

```

disp('构建优化模型...');

% 变量数量

num_x_vars = N * M; % x_ij 变量数量

num_y_vars = K;      % y_k 变量数量

num_z_vars = sum(sum(triu(Adj_matrix, 1))); % z_kk' 变量数量 (只计算上三角部分)

total_vars = num_x_vars + num_y_vars + num_z_vars;

% 变量索引映射函数

get_x_idx = @(i_idx, j_idx) (i_idx - 1) * M + j_idx;

get_y_idx = @(k_idx) num_x_vars + k_idx;

% 为 z 变量创建映射

z_var_indices = containers.Map('KeyType','char','ValueType','double');

current_z_idx = num_x_vars + num_y_vars;

for k1 = 1:K

    for k2 = (k1 + 1):K

        if Adj_matrix(k1, k2) == 1

            current_z_idx = current_z_idx + 1;

            z_var_indices(sprintf('%d_%d', k1, k2)) = current_z_idx;

        end

    end

end

end

get_z_idx = @(k1_idx, k2_idx) z_var_indices(sprintf('%d_%d', min(k1_idx, k2_idx),
max(k1_idx, k2_idx)));

% --- 目标函数 f (intlinprog 最小化 f*x) ---

f = zeros(total_vars, 1);

```

```

for row = 1:size(data1, 1)

    res_id = data1(row, 1);      % 居民 ID

    plot_id = data1(row, 2);     % 空置地块 ID

    is_eligible = data1(row, 3); % 是否合格

    cost = data1(row, 4);        % 补偿费用

    if is_eligible == 1 && resident_id_map.isKey(res_id) &&
empty_plot_id_map.isKey(plot_id)

        i_idx = resident_id_map(res_id);

        j_idx = empty_plot_id_map(plot_id);

        x_idx = get_x_idx(i_idx, j_idx);

        f(x_idx) = w4 * cost;

    end

end

for k_idx = 1:K

    y_idx = get_y_idx(k_idx);

    f(y_idx) = -w1 - w2 * Ak_total(k_idx);

end

for k1 = 1:K

    for k2 = (k1 + 1):K

        if Adj_matrix(k1, k2) == 1

            z_idx = get_z_idx(k1, k2);

            f(z_idx) = -w3;

        end

    end

end

```

```

end

end

% --- 约束条件  $A \cdot x \leq b$  ---

constraints = []; % 存储约束 [A_row, b_val]

constraint_idx = 0;

num_infeasible_constraints = 0;

for row = 1:size(data1, 1)

    is_eligible = data1(row, 3);

    if is_eligible == 0 % 如果是不合格的

        res_id = data1(row, 1);

        plot_id = data1(row, 2);

        if resident_id_map.isKey(res_id) && empty_plot_id_map.isKey(plot_id)

            i_idx = resident_id_map(res_id);

            j_idx = empty_plot_id_map(plot_id);

            x_idx = get_x_idx(i_idx, j_idx);

            A_row = zeros(1, total_vars);

            A_row(x_idx) = 1; %  $1 \cdot x_{ij}$ 

            b_val = 0; %  $\leq 0$ 

            constraint_idx = constraint_idx + 1;

            constraints(constraint_idx).A = A_row;

            constraints(constraint_idx).b = b_val;

            num_infeasible_constraints = num_infeasible_constraints + 1;

```

```

        end

    end

end

for i_idx = 1:N

    A_row = zeros(1, total_vars);

    for j_idx = 1:M

        x_idx = get_x_idx(i_idx, j_idx);

        A_row(x_idx) = 1;

    end

    constraint_idx = constraint_idx + 1;

    constraints(constraint_idx).A = A_row;

    constraints(constraint_idx).b = 1;

end

for j_idx = 1:M

    A_row = zeros(1, total_vars);

    for i_idx = 1:N

        x_idx = get_x_idx(i_idx, j_idx);

        A_row(x_idx) = 1;

    end

    constraint_idx = constraint_idx + 1;

    constraints(constraint_idx).A = A_row;

    constraints(constraint_idx).b = 1;

```

```

end

num_vacancy_constraints = 0;

for k_idx = 1:K

    residents_in_k = courtyard_residents{k_idx};

    if isempty(residents_in_k)

        continue; % 跳过没有居民的院落

    end

    y_idx = get_y_idx(k_idx);

    for i_idx_in_k = residents_in_k

        A_row = zeros(1, total_vars);

        A_row(y_idx) = 1; % +y_k

        for j_idx = 1:M

            x_idx = get_x_idx(i_idx_in_k, j_idx);

            A_row(x_idx) = -1; % -sum(x_ij)

        end

        constraint_idx = constraint_idx + 1;

        constraints(constraint_idx).A = A_row;

        constraints(constraint_idx).b = 0;

        num_vacancy_constraints = num_vacancy_constraints + 1;

    end

```

```

end

num_adj_constraints = 0;

for k1 = 1:K

    for k2 = (k1 + 1):K

        if Adj_matrix(k1, k2) == 1

            y1_idx = get_y_idx(k1);

            y2_idx = get_y_idx(k2);

            z_key = sprintf('%d_%d', min(k1, k2), max(k1, k2));

            if ~z_var_indices.isKey(z_key)

                continue; % 如果 Z 变量未创建则跳过

            end

            z_idx = get_z_idx(k1, k2);

            %  $z_{kk'} \leq y_k$ 

            A_row1 = zeros(1, total_vars); A_row1(z_idx) = 1; A_row1(y1_idx) = -1;

            constraint_idx = constraint_idx + 1; constraints(constraint_idx).A =
A_row1; constraints(constraint_idx).b = 0;

            %  $z_{kk'} \leq y_{k'}$ 

            A_row2 = zeros(1, total_vars); A_row2(z_idx) = 1; A_row2(y2_idx) = -1;

            constraint_idx = constraint_idx + 1; constraints(constraint_idx).A =
A_row2; constraints(constraint_idx).b = 0;

            %  $z_{kk'} \geq y_k + y_{k'} - 1$ 

            A_row3 = zeros(1, total_vars); A_row3(y1_idx) = -1; A_row3(y2_idx) = -
1; A_row3(z_idx) = 1;

            constraint_idx = constraint_idx + 1; constraints(constraint_idx).A =
A_row3; constraints(constraint_idx).b = -1;

```



```

        num_adj_constraints = num_adj_constraints + 3;

    end

end

end

% 6. 预算约束 (可选)
if ~isempty(B) && isnumeric(B) && B > 0

    A_row = zeros(1, total_vars);

    budget_constraint_terms = 0;

    for row = 1:size(data1, 1)

        res_id = data1(row, 1);

        plot_id = data1(row, 2);

        is_eligible = data1(row, 3);

        cost = data1(row, 4);

        if is_eligible == 1 && cost > 0 && resident_id_map.isKey(res_id) &&
empty_plot_id_map.isKey(plot_id)

            i_idx = resident_id_map(res_id);

            j_idx = empty_plot_id_map(plot_id);

            x_idx = get_x_idx(i_idx, j_idx);

            A_row(x_idx) = cost; % 成本只针对合格且有成本的搬迁

            budget_constraint_terms = budget_constraint_terms + 1;

        end

    end

end

```

```

if budget_constraint_terms > 0 % 仅当确实有成本项时才添加约束

    constraint_idx = constraint_idx + 1;

    constraints(constraint_idx).A = A_row;

    constraints(constraint_idx).b = B;

    fprintf('添加了 1 条预算约束 (<= %.2f), 包含 %d 个成本项\n', B,
budget_constraint_terms);

else

    warning('预算约束未添加, 因为所有合格搬迁成本都为 0。');

end

else

    disp('未设置预算约束或预算无效。');

end

% 组合 A 和 b 矩阵

A = vertcat(constraints.A);

b = vertcat(constraints.b);

for i = 1:constraint_idx

    A(i,:) = constraints(i).A;

    b(i) = constraints(i).b;

end

fprintf('总约束数量: %d\n', constraint_idx);

% --- 变量类型与边界 ---

intcon = 1:total_vars; % 所有变量都是整数 (二元)

```

```

lb = zeros(total_vars, 1); % 下界为 0

ub = ones(total_vars, 1); % 上界为 1


disp('模型构建完成。');

% --- 4. 求解模型 ---

disp('开始求解 MILP 问题...');

options = optimoptions('intlinprog', 'Display', 'iter');

[solution, fval, exitflag, output] = intlinprog(f, intcon, A, b, [], [], lb, ub, options);

% --- 5. 结果解析与展示 ---

disp('SUCCESFULLY。');

if exitflag > 0 % 找到了最优解或可行解

    fprintf('找到解! 目标函数值: %.4f\n', fval);

    fprintf('原始目标函数值 Z (最大化效益): %.4f\n', -fval);


% 解析结果

optimal_moves = [];

total_cost = 0;

vacated_courtyards = [];

adjacent_vacated_pairs = 0;

for i_idx = 1:N

    for j_idx = 1:M

        x_idx = get_x_idx(i_idx, j_idx);

        if solution(x_idx) > 0.5 % 考虑数值误差, 检查是否接近 1

            res_id = resident_ids(i_idx);

```

```

        plot_id = empty_plot_ids(j_idx);

        cost_row = find(data1(:,1) == res_id & data1(:,2) == plot_id);

        cost = 0;

        if ~isempty(cost_row)

            cost = data1(cost_row(1), 4);

            if cost > 200000

                cost = 200000;

            end

        end

        optimal_moves = [optimal_moves; res_id, plot_id, cost];

        total_cost = total_cost + cost;

    end

end

end

% 提取 y_k (腾退院落)

for k_idx = 1:K

    y_idx = get_y_idx(k_idx);

    if solution(y_idx) > 0.5

        vacated_courtyards = [vacated_courtyards; courtyard_ids(k_idx)];

    end

end

end

for k1 = 1:K

```

```

for k2 = (k1 + 1):K

    if Adj_matrix(k1, k2) == 1

        z_key = sprintf('%d_%d', min(k1, k2), max(k1, k2));

        if z_var_indices.isKey(z_key)

            z_idx = z_var_indices(z_key);

            if solution(z_idx) > 0.5

                adjacent_vacated_pairs = adjacent_vacated_pairs + 1;

            end

        end

    end

end

end

end

total_vacated_area = sum(Ak_total(ismember(courtyard_ids, vacated_courtyards)));

% 显示结果摘要

fprintf('\n--- 最优搬迁方案摘要 ---\n');

fprintf('总搬迁户数: %d\n', size(optimal_moves, 1));

fprintf('总补偿成本: %.2f\n', total_cost);

fprintf('完全腾退院落数量: %d\n', length(vacated_courtyards));

fprintf('完全腾退院落 ID: %s\n', num2str(vacated_courtyards));

fprintf('腾退的毗连院落对数量: %d\n', adjacent_vacated_pairs/2);

% 计算腾退后的空置地块数目

vacated_plot_count = 0;

```

```

for i = 1:length(vacated_courtyards)

    yard_id = vacated_courtyards(i);

    % 提取该院落的所有地块

    plots_in_yard = resident_plot_data(resident_plot_data(:,2) == yard_id, :);

    vacated_plot_count = vacated_plot_count + size(plots_in_yard, 1);

end

% 计算被使用的空置地块数目

used_empty_plots = size(optimal_moves, 1);

% 计算腾退后的空置地块数目

vacated_plot_count = 0;

for i = 1:length(vacated_courtyards)

    yard_id = vacated_courtyards(i);

    % 提取该院落的所有地块

    plots_in_yard = resident_plot_data(resident_plot_data(:,2) == yard_id, :);

    vacated_plot_count = vacated_plot_count + size(plots_in_yard, 1);

end

% 计算被使用的空置地块数目

used_empty_plots = size(optimal_moves, 1);

% 计算最终空置地块总数

total_empty_plots = length(empty_plot_ids) - used_empty_plots +
vacated_plot_count;

```

```

fprintf('完成搬迁后最终空置地块总数: %d\n', total_empty_plots);

% 计算最终空置地块总数

total_empty_plots = length(empty_plot_ids) - used_empty_plots +
vacated_plot_count;

fprintf('完成搬迁后最终空置地块总数: %d\n', total_empty_plots);

% 验证目标函数值

calculated_Z = w1 * length(vacated_courtyards) + ...

    w2 * total_vacated_area + ...

    w3 * adjacent_vacated_pairs - ...

    w4 * total_cost;

fprintf('手动验算目标函数 Z: %.4f\n', calculated_Z);

fprintf('(与求解器给出的 -fval=%.4f 是否一致?)\n', -fval);


% 保存结果到文件

if ~isempty(optimal_moves)

    result_table = array2table(optimal_moves, 'VariableNames', {'ResidentID',
'TargetPlotID', 'Cost'});

    writetable(result_table, '最优搬迁方案.xlsx');

    fprintf('最优搬迁方案已保存到 最优搬迁方案.xlsx\n');


% 保存腾退院落信息

vacated_info = [vacated_courtyards, arrayfun(@(k) Ak_total(courtyard_ids ==
k), vacated_courtyards)];

vacated_table = array2table(vacated_info, 'VariableNames', {'CourtyardID',
'Area'});

writetable(vacated_table, '腾退院落信息.xlsx');

```

```

        fprintf('腾退院落信息已保存到 腾退院落信息.xlsx\n');

    end

else

    if exitflag == 0

        warning('已达到最大迭代次数或时间限制，未找到最优解。');

    elseif exitflag == -2

        warning('问题无可行解。请检查约束条件和数据。');

    elseif exitflag == -3

        warning('问题无界。请检查目标函数和约束。');

    else

        warning('求解器返回了未知的退出标志: %d', exitflag);

        disp(output.message);

    end

end

save q2;

clear;clc

% 手动输入完全腾退的院落 ID

fully_vacated_courtyard_ids = [5 20 26 31 42 45 46 49 51 52 58 64 74 86 87 89 91 95 98
99 104 105 106];

% 读取原始地块数据

filename1 = '附件 1.xlsx';

data1 = readtable(filename1, 'ReadVariableNames', false);

plotsData = data1 {:, :};

plot_ids = plotsData(:, 1);

```



```

courtyard_ids = plotsData(:,2);

plot_areas = plotsData(:,3);

courtyard_areas = plotsData(:,4);

plot_orientations = plotsData(:,5);

is_occupied = plotsData(:,6);

filename3 = '最优搬迁方案.xlsx';

optimal_moves = readtable(filename3);

resident_ids = optimal_moves.ResidentID;

target_plot_ids = optimal_moves.TargetPlotID;

move_costs = load("q2.mat","total_cost");


load('data.mat');

adjacency_matrix = data;

% --- 2. 处理腾空院落信息 ---

disp('处理腾空院落信息...');


unique_courtyard_ids = unique(courtyard_ids);

courtyard_areas_map = containers.Map('KeyType', 'double', 'ValueType', 'double');

for i = 1:length(unique_courtyard_ids)

    courtyard_id = unique_courtyard_ids(i);

    idx = find(courtyard_ids == courtyard_id, 1, 'first');

    if ~isempty(idx)

        courtyard_areas_map(courtyard_id) = courtyard_areas(idx);

    end

```

```

end

plot_to_courtyard_map = containers.Map('KeyType', 'double', 'ValueType', 'double');
for i = 1:length(plot_ids)
    plot_to_courtyard_map(plot_ids(i)) = courtyard_ids(i);
end

courtyard_to_plots_map = containers.Map('KeyType', 'double', 'ValueType', 'any');
for i = 1:length(unique_courtyard_ids)
    courtyard_id = unique_courtyard_ids(i);
    courtyard_plots = plot_ids(courtyard_ids == courtyard_id);
    courtyard_to_plots_map(courtyard_id) = courtyard_plots;
end

plot_to_orientation_map = containers.Map('KeyType', 'double', 'ValueType', 'double');
for i = 1:length(plot_ids)
    plot_to_orientation_map(plot_ids(i)) = plot_orientations(i);
end

plot_to_area_map = containers.Map('KeyType', 'double', 'ValueType', 'double');
for i = 1:length(plot_ids)
    plot_to_area_map(plot_ids(i)) = plot_areas(i);
end

plot_to_occupied_map = containers.Map('KeyType', 'double', 'ValueType', 'double');

```

```

for i = 1:length(plot_ids)

    plot_to_occupied_map(plot_ids(i)) = is_occupied(i);
end

% --- 3. 计算所有成本 ---

disp('计算开发商成本...');

num_relocations = size(optimal_moves, 1);

communication_cost = 30000 * num_relocations;

compensation_cost = move_costs.total_cost;

area_loss_cost = 0;

for i = 1:num_relocations

    resident_id = resident_ids(i);

    target_id = target_plot_ids(i);

    if plot_to_area_map.isKey(resident_id) && plot_to_area_map.isKey(target_id)
    && ...

        plot_to_orientation_map.isKey(resident_id)

        original_area = plot_to_area_map(resident_id);

        target_area = plot_to_area_map(target_id);

        original_orientation = plot_to_orientation_map(resident_id);

        if target_area > original_area

            area_difference = target_area - original_area;

```

```

        % 根据原地块朝向计算机会成本

        if original_orientation == 3 % 南/北朝向

            daily_rate = 15;

        else % 东/西朝向

            daily_rate = 8;

        end

        area_loss_cost = area_loss_cost + (area_difference * daily_rate * 360);

    end

end

end

% 4. 时间成本: 4 个月/户搬迁（原址和目标地块无法利用）

time_cost = 0;

for i = 1:num_relocations

    resident_id = resident_ids(i);

    target_id = target_plot_ids(i);

    if plot_to_area_map.isKey(resident_id) && plot_to_area_map.isKey(target_id)
    && ...

        plot_to_orientation_map.isKey(resident_id) &&
        plot_to_orientation_map.isKey(target_id)

        original_area = plot_to_area_map(resident_id);

        target_area = plot_to_area_map(target_id);

        original_orientation = plot_to_orientation_map(resident_id);

```

```

target_orientation = plot_to_orientation_map(target_id);

if original_orientation == 3 % 南/北朝向

    daily_rate_original = 15;

else % 东/西朝向

    daily_rate_original = 8;

end

% 目标地块 4 个月无法利用的成本

if target_orientation == 3 % 南/北朝向

    daily_rate_target = 15;

else % 东/西朝向

    daily_rate_target = 8;

end

% 4 个月 = 120 天的租金损失

time_cost = time_cost + (original_area * daily_rate_original * 120);

time_cost = time_cost + (target_area * daily_rate_target * 120);

end

end

% 总成本

total_cost = communication_cost + compensation_cost + area_loss_cost + time_cost;

% --- 4. 计算收益 ---

```

```

disp('计算开发商收益...');

% 整院腾退的租金收益 (30 元/平米/天)

days_per_year = 360; % 1 年

daily_rate_courtyard = 30;

annual_rent_from_courtyards = 0;

for i = 1:length(fully_vacated_courtyard_ids)

    courtyard_id = fully_vacated_courtyard_ids(i);

    if courtyard_areas_map.isKey(courtyard_id)

        courtyard_area = courtyard_areas_map(courtyard_id);

        annual_rent_from_courtyards = annual_rent_from_courtyards +
(daily_rate_courtyard * courtyard_area * days_per_year);

    end

end

adjacent_bonus = 0;

adjacent_pairs = {};

courtyard_id_to_index = containers.Map('KeyType', 'double', 'ValueType', 'double');

for i = 1:length(unique_courtyard_ids)

    courtyard_id_to_index(unique_courtyard_ids(i)) = i;

end

for i = 1:length(fully_vacated_courtyard_ids)

    for j = i+1:length(fully_vacated_courtyard_ids)

        id1 = fully_vacated_courtyard_ids(i);

```

```

        id2 = fully_vacated_courtyard_ids(j);

        if courtyard_id_to_index.isKey(id1) && courtyard_id_to_index.isKey(id2)

            idx1 = courtyard_id_to_index(id1);

            idx2 = courtyard_id_to_index(id2);

            if idx1 <= size(adjacency_matrix,1) && idx2 <= size(adjacency_matrix,2)
&& adjacency_matrix(idx1, idx2) == 1

                if courtyard_areas_map.isKey(id1) &&
courtyard_areas_map.isKey(id2)

                    area1 = courtyard_areas_map(id1);

                    area2 = courtyard_areas_map(id2);

                    bonus = 0.2 * daily_rate_courtyard * (area1 + area2) *
days_per_year;

                    adjacent_bonus = adjacent_bonus + bonus;

                    adjacent_pairs{end+1} = [id1, id2];

                end

            end

        end

    end

end

% ===== 重新设计零散地块租金计算 =====

scattered_rent = 0;

% 1. 计算搬迁后的地块占用状态

relocated_plots = resident_ids; % 原居民地块（搬走后空置）

```

```

new_occupied_plots = target_plot_ids; % 新目标地块（搬迁后占用）

post_relocation_status = containers.Map('KeyType', 'double', 'ValueType', 'double');

for i = 1:length(plot_ids)

    post_relocation_status(plot_ids(i)) = plot_to_occupied_map(plot_ids(i));

end

for i = 1:length(relocated_plots)

    post_relocation_status(relocated_plots(i)) = 0;

    post_relocation_status(new_occupied_plots(i)) = 1;

end

for courtyard_id = unique_courtyard_ids'

    if ismember(courtyard_id, fully_vacated_courtyard_ids)

        continue;

    end

    if courtyard_to_plots_map.isKey(courtyard_id)

        courtyard_plots = courtyard_to_plots_map(courtyard_id);

        for plot_id = courtyard_plots'

            if post_relocation_status.isKey(plot_id) &&
post_relocation_status(plot_id) == 0

                if plot_to_area_map.isKey(plot_id) &&
plot_to_orientation_map.isKey(plot_id)

                    plot_area = plot_to_area_map(plot_id);

                    orientation = plot_to_orientation_map(plot_id);

                    if orientation == 3 % 南/北朝向

                        daily_rate = 15;

                    else % 东/西朝向

```



```

        daily_rate = 8;

        end

        % 计算年租金

        scattered_rent = scattered_rent + (daily_rate * plot_area *
days_per_year);

        end

    end

end

end

end

end

% 计算总年租金收入

total_annual_rent = annual_rent_from_courtyards + adjacent_bonus + scattered_rent;

% 计算净利润

net_profit = total_annual_rent - total_cost;

% 计算腾退院落的总面积

total_vacated_area = 0;

for i = 1:length(fully_vacated_courtyard_ids)

    courtyard_id = fully_vacated_courtyard_ids(i);

    if courtyard_areas_map.isKey(courtyard_id)

        total_vacated_area = total_vacated_area + courtyard_areas_map(courtyard_id);

```

```

        end

    end

% --- 5. 显示结果 ---

fprintf('\n=== 问题二最优方案开发商经济效益分析 (1 年) ===\n');

fprintf('完全腾退院落数量: %d\n', length(fully_vacated_courtyard_ids));

fprintf('相邻腾退院落对数: %d\n', length(adjacent_pairs));

fprintf('腾退院落的总面积: %.2f 平方米\n', total_vacated_area);

fprintf('搬迁户数: %d\n', num_relocations);


fprintf('\n--- 成本分析 ---\n');

fprintf('1. 沟通成本: %.2f 元\n', communication_cost);

fprintf('2. 补偿费用(含装修): %.2f 元\n', compensation_cost);

fprintf('3. 面积损失成本: %.2f 元\n', area_loss_cost);

fprintf('4. 时间成本: %.2f 元\n', time_cost);

fprintf('总成本: %.2f 元\n', total_cost);

% --- 6. 可视化分析 ---

disp('生成可视化图表...');


% 1. 成本构成分析

cost_labels = {'沟通成本', '补偿费用', '面积损失', '时间成本'};

cost_values = [communication_cost, compensation_cost, area_loss_cost, time_cost];


figure;

```

```

subplot(1,2,1);

pie(cost_values, cost_labels);

title('成本构成分析 (1 年)');


% 2. 收入来源分解

income_labels = {'整院租金', '毗邻奖励', '零散地块'};

income_values = [annual_rent_from_courtyards, adjacent_bonus, scattered_rent];


subplot(1,2,2);

pie(income_values, income_labels);

title('收入来源分解 (1 年)');

set(gcf,'Position',[200 200 1200 500]);


% 3. 利润对比可视化

figure;

profit_matrix = [total_cost, total_annual_rent, net_profit];

b = bar(profit_matrix);

set(gca,'XTickLabel',{'总成本','总收入','净利润'});

ylabel('金额 (元)');

title('经济效益对比');

grid on;


% 在柱子上显示数值

xtips = b.XEndPoints;

ytips = b.YEndPoints;

```

```

labels = string(b.YData);

text(xtips, ytips, labels,'HorizontalAlignment','center',...

    'VerticalAlignment','bottom')

% 4. 腾退院落毗邻关系网络图

figure;

vacated_indices = arrayfun(@(x)courtyard_id_to_index(x), fully_vacated_courtyard_ids);

% 创建简化毗邻矩阵（仅显示腾退院落）

adj_vacated = adjacency_matrix(vacated_indices, vacated_indices);

G = graph(adj_vacated, cellstr(num2str(fully_vacated_courtyard_ids)));

% 检测毗邻对

edge_weights = zeros(numedges(G),1);

for i = 1:length(adjacent_pairs)

    [~,id1] = ismember(adjacent_pairs{i}(1),fully_vacated_courtyard_ids);

    [~,id2] = ismember(adjacent_pairs{i}(2),fully_vacated_courtyard_ids);

    edge_idx = findedge(G, num2str(adjacent_pairs{i}(1)),
num2str(adjacent_pairs{i}(2)));

    if edge_idx > 0

        edge_weights(edge_idx) = 1;

    end

end

end

```

```

% 绘制网络图

h = plot(G, 'Layout','force','EdgeLabel',{},'NodeColor',[0.2 0.6 0.8]);

highlight(h, 'Edges', find(edge_weights), 'LineWidth', 2, 'LineStyle','-', 'EdgeColor','r');

title('腾退院落毗邻关系网络');

xlabel('红色边表示产生额外收益的毗邻对');


% 5. 各院落面积贡献分析

vacated_areas = zeros(size(fully_vacated_courtyard_ids));

for i = 1:length(fully_vacated_courtyard_ids)

    vacated_areas(i) = courtyard_areas_map(fully_vacated_courtyard_ids(i));

end


figure;

[sorted_areas, idx] = sort(vacated_areas,'descend');

sorted_ids = fully_vacated_courtyard_ids(idx);


barh(sorted_areas);

set(gca,'YTick',1:length(sorted_ids),'YTickLabel',sorted_ids);

xlabel('院落面积 (平方米)');

ylabel('院落 ID');

title('腾退院落面积排序');

grid on;


disp('可视化图表生成完成。');

```

python 部分:

```
# ----- 文件 & 列名配置 -----
PLOT_FILE = r"C:\Users\TD\Documents\WeChat
Files\wxid_emdbqb5j9k6z22\FileStorage\File\2025-
04\2025MathorCup\2025MathorCup\B\搬迁后数据.xlsx"      #按照文件保存路径
读取搬迁后数据.xlsx
DIK_FILE = r"C:\Users\TD\Documents\WeChat
Files\wxid_emdbqb5j9k6z22\FileStorage\File\2025-
04\2025MathorCup\2025MathorCup\B\d_ik.xlsx"      #按照文件保存路径读取
d_ik.xlsx
SHEET_PLOT = 0
SHEET_DIK = 0

COL_PLOT = "plotID"      # 地块编号
COL_COURT = "courtID"    # 院落编号
COL_AREA = "plotArea"    # 地块面积 (m²)
COL_DIR = "direction"    # 'south'/'north'/'east'/'west'
COL_VACANT = "isVacant"  # 0=空置 1=有住户
COL_CA = "courtArea"     # 院落总面积

UNIT_PRICE = {"south":15,"north":15,"east":8,"west":8}
WHOLE_UNIT, NEI_MULT = 30, 1.2
DAY_10Y = 365 * 10
# -----

# 1. 读数据 -----
plot_df = pd.read_excel(PLOT_FILE)
d_ik = pd.read_excel(DIK_FILE, header=None).to_numpy()

# 2. 方向统一为小写 -----
plot_df[COL_DIR] = plot_df[COL_DIR].str.lower().str.strip()

# 3. 生成“院落→地块列表”映射 -----
court_to_plots = (plot_df.groupby(COL_COURT)[COL_PLOT]
                  .apply(list).to_dict())

# 4. 判定整院 -----
# 整院 = 该院落所有地块 isVacant == 0
court_to_whole = {cid: (plot_df.loc[plot_df[COL_COURT]==cid,
COL_VACANT]==0).all()
                  for cid in court_to_plots}

whole_courts = [cid for cid,flag in court_to_whole.items() if flag]

# 5. 毗邻判断与毗邻整院判定 -----
# 假设 d_ik 的行/列顺序与“院落编号升序”完全一致；若不是请先重排
```

```

court_sorted = sorted(plot_df[COL_COURT].unique())
court2idx     = {cid:i for i,cid in enumerate(court_sorted)}

# —— 所有“有邻居”的院落 ID（不分整院） —— NEW
all_adj_flag   = d_ik.any(axis=1)           # bool 行向量
all_adj_courts = [court_sorted[i] for i,flag in enumerate(all_adj_flag) if flag]

whole_idx = [court2idx[cid] for cid in whole_courts]
submat     = d_ik[np.ix_(whole_idx, whole_idx)].copy()
np.fill_diagonal(submat, 0)

whole_nei_flag = submat.any(axis=1)          # 布尔数组
court_is_nei   = dict(zip(whole_courts, whole_nei_flag))

nei_whole_cnt  = int(whole_nei_flag.sum())
whole_cnt      = len(whole_courts)
nonei_whole_cnt = whole_cnt - nei_whole_cnt
nei_whole_ids  = [cid for cid,flag in court_is_nei.items() if flag]

# 6. 分散空置地块 -----
mask_disp = (plot_df[COL_VACANT]==0) &
(~plot_df[COL_COURT].isin(whole_courts))
disp_df    = plot_df[mask_disp]
disp_cnt   = len(disp_df)

# 7. 收益计算（拆分三类） -----
daily_whole_base = 0.0      # 整院基础租金
daily_nei_extra  = 0.0      # 毗邻整院额外租金
daily_disp_income = 0.0     # 分散空置地块租金
whole_area_sum   = 0.0      # 整院总面积

court_area = plot_df.groupby(COL_COURT)[COL_CA].first().to_dict()

# 7-A 整院
for cid in whole_courts:
    area = court_area[cid]
    base = WHOLE_UNIT * area          # 30×area
    daily_whole_base += base
    whole_area_sum   += area
    if court_is_nei[cid]:              # 有毗邻整院 → 额外 0.2×30×area
        daily_nei_extra += (NEI_MULT - 1) * base    # (1.2-1)=0.2

# 7-B 分散空置
daily_disp_income = (
    disp_df[COL_AREA] *
    disp_df[COL_DIR].map(lambda d: UNIT_PRICE.get(d, 8))

```

```

).sum()

# 合计日收益
daily_income = daily_whole_base + daily_nei_extra + daily_disp_income
income_10y    = daily_income * DAY_10Y

# 各部分十年收益
whole_income_10y = daily_whole_base * DAY_10Y
extra_income_10y = daily_nei_extra * DAY_10Y
disp_income_10y  = daily_disp_income * DAY_10Y

# 8. 打印结果 -----
print(f'每日租金收益(不搬迁): {daily_income:,.0f} 元')
print(f'十年总收益(不搬迁):    {income_10y:,.0f} 元\n')

print("整院院落 ID 列表:", whole_courts)
print("毗邻院落 ID 列表:", all_adj_courts)
print("毗邻整院 ID 列表:", nei_whole_ids)
print(f'整院院落总面积: {whole_area_sum:,.0f} m²\n')

print("—— 十年收益拆分 ——")
print(f' 1) 整院租金收益           : {whole_income_10y:,.0f} 元')
print(f' 2) 毗邻院落额外租金收益   : {extra_income_10y:,.0f} 元')
print(f' 3) 分散空置地块租金收益   : {disp_income_10y:,.0f} 元')
print(f' 合计                       :
{whole_income_10y+extra_income_10y+disp_income_10y:,.0f} 元')

summary = {
    "整院院落数"           : whole_cnt,
    "毗邻整院数"           : nei_whole_cnt,
    "非毗邻整院数"         : nonei_whole_cnt,
    "分散空置地块数"       : disp_cnt,
    "整院院落总面积"       : whole_area_sum,
    "整院十年收益"         : whole_income_10y,
    "毗邻加成十年收益"     : extra_income_10y,
    "分散十年收益"         : disp_income_10y,
    "income_10y"           : income_10y
}
print("\n 汇总信息:", summary)

```


附录 3：问题三算法程序

程序编号	3	程序说明	考虑 m 的 MILP 求解模型	编程软件	Matlab+python
<pre> Python 部分: # check_court_neighbors_v2.py import pandas as pd import numpy as np from pathlib import Path # ----- 文件路径 (自行修改) ----- DIK_FILE = (r"C:\Users\TD\Documents\WeChat Files\wxid_emdbqb5j9k6z22\FileStorage\File\2025- 04\2025MathorCup\2025MathorCup\B\d_ik.xlsx") QUERY_FILE = (r"C:\Users\TD\Documents\WeChat Files\wxid_emdbqb5j9k6z22\FileStorage\File\2025- 04\2025MathorCup\2025MathorCup\B\院落查询 ID 列表.xlsx") # ← 放你的 query_ids 表 QUERY_COL = "courtID" # Excel 中存放院落 ID 的列名 # 1. 读邻接矩阵 & 查询 ID ----- d_ik = pd.read_excel(DIK_FILE, header=None).to_numpy() query_ids = pd.read_excel(QUERY_FILE)[QUERY_COL].dropna().astype(int).tolist() # 2. 院落 ID ↔ 索引映射 (假设行列升序 1..N; 若不是请手动列出) court_ids = np.arange(1, d_ik.shape[0] + 1) # [1,2,...,N] idx_map = {cid:i for i,cid in enumerate(court_ids)} # 3. 逐院落输出邻接 ID ----- print("=== 单个院落的邻接列表 ===") for cid in query_ids: if cid not in idx_map: print(f"[院落 {cid}] 不在矩阵索引中, 跳过。") continue i = idx_map[cid] neighbor_mask = (d_ik[i, :] d_ik[:, i]).astype(bool) neighbor_mask[i] = False neighbors = court_ids[neighbor_mask] print(f"[院落 {cid}] 毗邻院落 ID: {neighbors.tolist() if neighbors.size else '无'}") # 4. 统计 query_ids 内部互毗邻 ----- valid_pairs = [(a,b) for i,a in enumerate(query_ids) for b in query_ids[i+1:] if a in idx_map and b in idx_map </pre>					

```

                                and (d_ik[idx_map[a], idx_map[b]] or
                                    d_ik[idx_map[b], idx_map[a]])

mutual_ids = sorted({x for pair in valid_pairs for x in pair})
mutual_cnt = len(mutual_ids)

print("\n 内部互为毗邻的院落 ID 集合 ===")
print(mutual_ids if mutual_ids else "无互相毗邻的院落")
print(f"\n 毗邻院落数: {mutual_cnt}")
# check_court_neighbors.py
import pandas as pd
import numpy as np
from pathlib import Path

# ----- 文件 & 列名配置 -----
PLOT_FILE = r"C:\Users\TD\Documents\WeChat
Files\wxid_emdbqb5j9k6z22\FileStorage\File\2025-
04\2025MathorCup\2025MathorCup\B\1.xlsx" #按照文件保存路径读取 1.xlsx
DIK_FILE = r"C:\Users\TD\Documents\WeChat
Files\wxid_emdbqb5j9k6z22\FileStorage\File\2025-
04\2025MathorCup\2025MathorCup\B\d_ik.xlsx" #按照文件保存路径读取
d_ik.xlsx
SHEET_PLOT = 0
SHEET_DIK = 0

COL_PLOT = "plotID" # 地块编号
COL_COURT = "courtID" # 院落编号
COL_AREA = "plotArea" # 地块面积 (m²)
COL_DIR = "direction" # 'south'/'north'/'east'/'west'
COL_VACANT = "isVacant" # 0=空置 1=有住户
COL_CA = "courtArea" # 院落总面积

UNIT_PRICE = {"south":15,"north":15,"east":8,"west":8}
WHOLE_UNIT, NEI_MULT = 30, 1.2
DAY_10Y = 365 * 10
# -----

# 1. 读数据 -----
plot_df = pd.read_excel(PLOT_FILE)
d_ik = pd.read_excel(DIK_FILE, header=None).to_numpy()

# 2. 方向统一为小写 -----
plot_df[COL_DIR] = plot_df[COL_DIR].str.lower().str.strip()

# 3. 生成“院落→地块列表”映射 -----
court_to_plots = (plot_df.groupby(COL_COURT)[COL_PLOT]

```

```

        .apply(list).to_dict())

# 4. 判定整院 -----
# 整院 = 该院落所有地块 isVacant == 0
court_to_whole = {cid: (plot_df.loc[plot_df[COL_COURT]==cid,
COL_VACANT]==0).all()
                  for cid in court_to_plots}

whole_courts = [cid for cid,flag in court_to_whole.items() if flag]

# 5. 毗邻判断与毗邻整院判定 -----
# 假设 d_ik 的行/列顺序与“院落编号升序”完全一致；若不是请先重排
court_sorted = sorted(plot_df[COL_COURT].unique())
court2idx = {cid:i for i,cid in enumerate(court_sorted)}

# —— 所有“有邻居”的院落 ID（不分整院） —— NEW
all_adj_flag = d_ik.any(axis=1) # bool 行向量
all_adj_courts = [court_sorted[i] for i,flag in enumerate(all_adj_flag) if flag]

whole_idx = [court2idx[cid] for cid in whole_courts]
submat = d_ik[np.ix_(whole_idx, whole_idx)].copy()
np.fill_diagonal(submat, 0)

whole_nei_flag = submat.any(axis=1) # 布尔数组
court_is_nei = dict(zip(whole_courts, whole_nei_flag))

nei_whole_cnt = int(whole_nei_flag.sum())
whole_cnt = len(whole_courts)
nonei_whole_cnt = whole_cnt - nei_whole_cnt
nei_whole_ids = [cid for cid,flag in court_is_nei.items() if flag]

# 6. 分散空置地块 -----
mask_disp = (plot_df[COL_VACANT]==0) &
(~plot_df[COL_COURT].isin(whole_courts))
disp_df = plot_df[mask_disp]
disp_cnt = len(disp_df)

# 7. 收益计算（拆分三类） -----
daily_whole_base = 0.0 # 整院基础租金
daily_nei_extra = 0.0 # 毗邻整院额外租金
daily_disp_income = 0.0 # 分散空置地块租金
whole_area_sum = 0.0 # 整院总面积

court_area = plot_df.groupby(COL_COURT)[COL_CA].first().to_dict()

# 7-A 整院

```

```

for cid in whole_courts:
    area = court_area[cid]
    base = WHOLE_UNIT * area                # 30×area
    daily_whole_base += base
    whole_area_sum += area
    if court_is_nei[cid]:                    # 有毗邻整院 → 额外 0.2×30×area
        daily_nei_extra += (NEI_MULT - 1) * base    # (1.2-1)=0.2

# 7-B 分散空置
daily_disp_income = (
    disp_df[COL_AREA] *
    disp_df[COL_DIR].map(lambda d: UNIT_PRICE.get(d, 8))
).sum()

# 合计日收益
daily_income = daily_whole_base + daily_nei_extra + daily_disp_income
income_10y = daily_income * DAY_10Y

# 各部分十年收益
whole_income_10y = daily_whole_base * DAY_10Y
extra_income_10y = daily_nei_extra * DAY_10Y
disp_income_10y = daily_disp_income * DAY_10Y

# 8. 打印结果 -----
print(f'每日租金收益(不搬迁): {daily_income:,.0f} 元')
print(f'十年总收益(不搬迁): {income_10y:,.0f} 元\n')

print("整院院落 ID 列表:", whole_courts)
print("毗邻院落 ID 列表:", all_adj_courts)
print("毗邻整院 ID 列表:", nei_whole_ids)
print(f'整院院落总面积: {whole_area_sum:,.0f} m²\n')

print("—— 十年收益拆分 ——")
print(f'1) 整院租金收益 : {whole_income_10y:,.0f} 元')
print(f'2) 毗邻院落额外租金收益 : {extra_income_10y:,.0f} 元')
print(f'3) 分散空置地块租金收益 : {disp_income_10y:,.0f} 元')
print(f'合 计 : {whole_income_10y+extra_income_10y+disp_income_10y:,.0f} 元')

summary = {
    "整院院落数" : whole_cnt,
    "毗邻整院数" : nei_whole_cnt,
    "非毗邻整院数" : nonei_whole_cnt,
    "分散空置地块数" : disp_cnt,
    "整院院落总面积" : whole_area_sum,
    "整院十年收益" : whole_income_10y,

```

```

    "毗邻加成十年收益" : extra_income_10y,
    "分散十年收益"      : disp_income_10y,
    "income_10y"        : income_10y
}
print("\n 汇总信息:", summary)
Matlab 部分:
% MATLAB 代码求解性价增益拐点问题
clear;clc

%% 基础数据准备
% 加载数据
result1_file = 'result1.xlsx';
data1 = readmatrix(result1_file);
plot_data_file = '附件 1.xlsx';
data2 = readmatrix(plot_data_file);
load("data.mat");
Adj_matrix = data;

% 参数设置
unit_rent = 100; % 假设单位面积年租金(元/平方米)
years = 10;      % 计算周期
min_m = 20;      % 最低性价比要求

%% 数据预处理
% 提取居民和院落信息
resident_ids = unique(data1(:,1));
empty_plot_ids = unique(data1(:,2));
N = length(resident_ids);
M = length(empty_plot_ids);

% 提取有居民的院落数据
resident_mask = data2(:,6) == 1;
resident_plot_data = data2(resident_mask,:);
courtyard_ids = unique(resident_plot_data(:,2));
K = length(courtyard_ids);

% 建立映射表
resident_id_map = containers.Map(resident_ids, 1:N);
empty_plot_id_map = containers.Map(empty_plot_ids, 1:M);
courtyard_id_map = containers.Map(courtyard_ids, 1:K);

% 计算院落面积
Ak_total = zeros(1,K);
for k = 1:K
    yard_id = courtyard_ids(k);
    Ak_total(k) = sum(resident_plot_data(resident_plot_data(:,2)==yard_id,3));

```

```

end

%% 模型参数配置
model_params = struct();
model_params.N = N;
model_params.M = M;
model_params.K = K;
model_params.Adj = Adj_matrix;
model_params.rent = unit_rent*years; % 十年期租金系数
model_params.min_m = min_m;

%% 迭代求解不同腾退规模下的最优解
max_courtyards = K;
m_values = zeros(1,max_courtyards);
solutions = cell(1,max_courtyards);

for n = 1:max_courtyards
    % 构建优化模型
    [f, A, b, Aeq, beq, var_info] = build_optimization_model(...
        data1, resident_plot_data, model_params, n);

    % 求解整数规划
    [sol, ~, exitflag] = intlinprog(f, var_info.intcon, A, b, Aeq, beq,...
        var_info.lb, var_info.ub, optimoptions('intlinprog','Display','off'));

    % 结果处理
    if exitflag > 0
        [m, cost, rent_income] = calculate_metrics(sol, var_info, Ak_total,
model_params);
        m_values(n) = m;
        solutions{n} = struct('solution',sol,'cost',cost,'income',rent_income);
    else
        m_values(n) = 0;
    end
    fprintf('腾退院落数: %d, 性价比 m=%.2f\n',n,m_values(n));
end

%% 结果分析与可视化
% 绘制性价比曲线
figure;
plot(1:max_courtyards, m_values, 'b-o','LineWidth',1.5);
xlabel('腾退院落数量');
ylabel('性价比 m');
title('腾退规模与性价比关系');
grid on;

```

```

% 寻找拐点
[peak_m, peak_idx] = max(m_values);
diff_m = diff(m_values);
inflection_points = find(diff(diff_m) < 0) + 1;

% 结果输出
if any(m_values >= min_m)
    valid_idx = find(m_values >= min_m, 1, 'last');
    fprintf('\n=== 拐点分析结果 ===\n');

    if ~isempty(inflection_points)
        [~, max_infl] = min(diff_m(inflection_points));
        best_idx = inflection_points(max_infl);
        fprintf('发现拐点位于腾退 %d 个院落处，\n', best_idx, m_values(best_idx));
    else
        best_idx = peak_idx;
        fprintf('未发现明显拐点，最大 m=%0.2f 出现在腾退 %d 个院落\n', peak_m, peak_idx);
    end

% 提取最佳方案
best_solution = solutions{best_idx};
[movement, vacated] = parse_solution(best_solution.solution, var_info, ...
    resident_ids, empty_plot_ids, courtyard_ids, resident_plot_data);

% 显示详细信息
fprintf('\n=== 最优搬迁方案 ===\n');
fprintf('腾退院落数量: %d\n', length(vacated));
fprintf('总补偿成本: %0.2f 元\n', best_solution.cost);
fprintf('预期租金收益: %0.2f 元\n', best_solution.income);
fprintf('盈利: %0.2f 元\n', best_solution.income - best_solution.cost);

% 保存结果
save_solution_details(movement, vacated, Ak_total, courtyard_ids);
else
    fprintf('\n警告：所有方案均未达到 m>=20 的要求\n');
    fprintf('当前最大 m=%0.2f，出现在腾退 %d 个院落\n', peak_m, peak_idx);
end

%% 模型构造函数
function [f, A, b, Aeq, beq, var_info] = build_optimization_model(...
    data1, resident_data, params, n)

% 变量初始化

```

```

num_x = params.N * params.M;
num_y = params.K;
num_z = sum(sum(triu(params.Adj,1)));
total_vars = num_x + num_y + num_z;

% 变量索引系统
var_info = struct();
var_info.intcon = 1:total_vars;
var_info.lb = zeros(total_vars,1);
var_info.ub = ones(total_vars,1);
var_info.x_idx = @(i,j) (i-1)*params.M + j;
var_info.y_idx = @(k) num_x + k;

% 目标函数：最大化租金收入 - m*成本
f = zeros(total_vars,1);

% 成本项
for row = 1:size(data1,1)
    if data1(row,3) == 1 % 合格搬迁
        i = data1(row,1);
        j = data1(row,2);
        f(var_info.x_idx(i,j)) = -params.min_m * data1(row,4); % 最小化成本项
    end
end

% 收益项（与腾退面积成正比）
for k = 1:params.K
    f(var_info.y_idx(k)) = params.rent * Ak_total(k); % 最大化租金收益
end

% 约束条件构建
[A, b] = build_basic_constraints(data1, params, var_info);

% 添加腾退数量约束
Aeq = zeros(1,total_vars);
for k = 1:params.K
    Aeq(1,var_info.y_idx(k)) = 1;
end
beq = n;
end

%% 基础约束构建
function [A, b] = build_basic_constraints(data1, params, var_info)
    constraints = [];

    % 原始约束构建逻辑（搬迁资格、单地块分配等）

```



```

% ...此处省略具体实现，保留原有约束生成逻辑...

% 组合约束矩阵
A = vertcat(constraints.A);
b = vertcat(constraints.b);
end

%% 指标计算函数
function [m, total_cost, rent_income] = calculate_metrics(sol, var_info, Ak, params)
% 提取搬迁成本
total_cost = sum(sol(1:var_info.N*var_info.M) .* ...
    (data1(data1(:,3)==1,4)' .* params.min_m));

% 计算租金收益
y_vars = sol(var_info.N*var_info.M+1:var_info.N*var_info.M+var_info.K);
rent_income = sum(y_vars .* Ak * params.rent);

% 计算性价比
m = rent_income / total_cost;
end

%% 解决方案解析
function [movement, vacated] = parse_solution(sol, var_info, res_ids, emp_ids, yard_ids,
plot_data)
% 提取搬迁方案
movement = [];
for i = 1:var_info.N
    for j = 1:var_info.M
        if sol(var_info.x_idx(i,j)) > 0.5
            movement = [movement; res_ids(i), emp_ids(j)];
        end
    end
end
end

% 提取腾退院落
vacated = yard_ids(logical(sol(var_info.y_idx(1):var_info.y_idx(end))));

% 计算腾退后的空置地块数目
vacated_plot_count = 0;
for i = 1:length(vacated_courtyards)
    yard_id = vacated_courtyards(i);
    % 提取该院落的所有地块
    plots_in_yard = resident_plot_data(resident_plot_data(:,2) == yard_id, :);
    vacated_plot_count = vacated_plot_count + size(plots_in_yard, 1);
end
end

```

```

end

%% 结果保存函数
function save_solution_details(movement, vacated, areas, yard_ids)
    % 保存搬迁方案
    writetable(array2table(movement,'VariableNames',{'原居民','迁入地块'}),...
        'optimal_moves.xlsx');
    % 保存腾退院落信息
    yard_info = [vacated', arrayfun(@(x)areas(yard_ids==x), vacated')];
    writetable(array2table(yard_info,'VariableNames',{'院落 ID','面积'}),...
        'vacated_yards.xlsx');
end

%% 读取数据
tic
clear;clc
% 读取附件 1.xlsx
filename1 = '附件 1.xlsx';
data1 = readtable(filename1, 'ReadVariableNames', false);
plotsData = data1{:, :};
plot_ids = plotsData(:,1);           % 地块 ID
courtyard_ids = plotsData(:,2);      % 院落 ID
plot_areas = plotsData(:,3);         % 地块面积
courtyard_areas = plotsData(:,4);    % 院落面积
plot_orientations = plotsData(:,5);  % 地块朝向(3=南/北, 2=东, 1=西)
is_occupied = plotsData(:,6);        % 是否有住户(1=有, 0=无)

% 读取 result1.xlsx
filename2 = 'result1.xlsx';
data2 = readtable(filename2, 'ReadVariableNames', false);
resultData = data2{:, :};
resident_plots = resultData(:,1);    % 居民所在地块 ID
vacant_plots = resultData(:,2);      % 空置地块 ID
is_eligible = resultData(:,3);       % 是否合格
compensation_costs = resultData(:,4); % 补偿费用

% 读取毗邻矩阵
load('data.mat'); % 假设矩阵名为 adjacency_matrix
adjacency_matrix = data; % 107x107 毗邻矩阵

% 设置 m 的范围和步长
m_values = 15:1:25;
%% 处理数据

% 获取所有居民 ID (已占用的地块 ID)
occupied_plots = plot_ids(is_occupied == 1);

```

```

num_residents = length(occupied_plots);

% 获取所有空置地块 ID
vacant_plot_ids = plot_ids(is_occupied == 0);
num_vacant_plots = length(vacant_plot_ids);

% 获取所有院落 ID
unique_courtyard_ids = unique(courtyard_ids);
num_courtyards = length(unique_courtyard_ids);

% 构建院落与地块的映射关系
courtyard_plots = cell(num_courtyards, 1);
courtyard_occupied_plots = cell(num_courtyards, 1);
courtyard_total_areas = zeros(num_courtyards, 1);

for i = 1:num_courtyards
    courtyard_id = unique_courtyard_ids(i);
    courtyard_plots{i} = plot_ids(courtyard_ids == courtyard_id);
    courtyard_occupied_plots{i} = intersect(courtyard_plots{i}, occupied_plots);

    % 计算院落总面积
    courtyard_idx = find(courtyard_ids == courtyard_id, 1);
    courtyard_total_areas(i) = courtyard_areas(courtyard_idx);
end

% 构建居民与空置地块的映射关系
eligible_map = zeros(num_residents, num_vacant_plots);
cost_map = zeros(num_residents, num_vacant_plots);

for i = 1:size(resultData, 1)
    resident_idx = find(occupied_plots == resident_plots(i));
    vacant_idx = find(vacant_plot_ids == vacant_plots(i));

    if ~isempty(resident_idx) && ~isempty(vacant_idx)
        eligible_map(resident_idx, vacant_idx) = is_eligible(i);
        cost_map(resident_idx, vacant_idx) = compensation_costs(i);
    end
end

%% 计算租金收益

% 零散地块日租金: 东/西向 8 元/平米/天, 南/北向 15 元/平米/天
% 整院腾退日租金: 30 元/平米/天
% 相邻整院腾退日租金: 36 元/平米/天

% 计算每个院落腾退后的年租金收益

```

```

courtyard_annual_rent = zeros(num_courtyards, 1);

for i = 1:num_courtyards
    courtyard_id = unique_courtyard_ids(i);
    courtyard_area = courtyard_total_areas(i);

    % 整院腾退的年租金 (每年按 360 天计算)
    courtyard_annual_rent(i) = 30 * courtyard_area * 360;
end

%% 构建和求解带参数 m 的整数线性规划模型，探索拐点

num_m = length(m_values);
objective_values = zeros(num_m, 1);
total_cost_values = zeros(num_m, 1);
total_rent_values = zeros(num_m, 1);
evacuated_courtyards = zeros(num_m, 1);
adjacent_pairs = zeros(num_m, 1);

% 构建模型的基本框架
num_residents_vars = num_residents * num_vacant_plots;
num_courtyard_vars = num_courtyards;
num_adjacent_vars = sum(sum(adjacency_matrix)) / 2;
total_vars = num_residents_vars + num_courtyard_vars + num_adjacent_vars;

% 构建目标函数系数
f = zeros(total_vars, 1);

% 居民搬迁变量的目标系数为 0
% 院落腾退变量的目标系数为其租金收益
for i = 1:num_courtyards
    f(num_residents_vars + i) = courtyard_annual_rent(i) * 10; % 10 年收益
end

% 相邻院落对变量的目标系数为额外收益 (20%加成)
adjacent_offset = num_residents_vars + num_courtyard_vars;
adjacent_idx = 1;
for i = 1:num_courtyards
    for j = i+1:num_courtyards
        if adjacency_matrix(i,j) == 1
            % 相邻院落额外收益 = 0.2 * (院落 i 年租金 + 院落 j 年租金) * 10 年
            f(adjacent_offset + adjacent_idx) = 0.2 * (courtyard_annual_rent(i) +
courtyard_annual_rent(j)) * 10;
            adjacent_idx = adjacent_idx + 1;
        end
    end
end
end

```

```

end

% 构建约束条件矩阵
% 1. 居民搬迁可行性约束:  $x_{ij} \leq e_{ij}$ 
% 2. 每个居民最多搬到一个地块:  $\sum_j (x_{ij}) \leq 1$ 
% 3. 每个空置地块最多被一个居民占用:  $\sum_i (x_{ij}) \leq 1$ 
% 4. 院落腾退约束:  $y_k \leq \sum_j (x_{ij}) / |R_k|$  for all  $i$  in  $R_k$ 
% 5. 相邻院落对约束:  $z_{kk'} \leq y_k, z_{kk'} \leq y_{k'}$ 
% 6. 性价比约束: 十年总租金收益  $\geq m * \text{总投入}$ 

% 为每个 m 值求解模型
for m_idx = 1:num_m
    m = m_values(m_idx);
    disp(['求解 m = ', num2str(m)]);

    % 构建约束矩阵和约束向量 (根据 m 值更新性价比约束)
    [A, b, Aeq, beq, lb, ub] = buildConstraints(num_residents, num_vacant_plots,
num_courtyards, adjacency_matrix, ...
        eligible_map, courtyard_occupied_plots, cost_map, f, m, occupied_plots);

    % 设置整数约束
    intcon = 1:total_vars;

    % 求解整数线性规划
    options = optimoptions('intlinprog', 'Display', 'off');
    [x, fval, exitflag] = intlinprog(-f, intcon, A, b, Aeq, beq, lb, ub, options);

    if exitflag > 0
        % 解析结果
        objective_values(m_idx) = -fval; % 最大化问题，所以取负

        % 计算总成本
        resident_vars = x(1:num_residents_vars);
        resident_vars_reshaped = reshape(resident_vars, num_residents,
num_vacant_plots);
        total_cost = 0;
        for i = 1:num_residents
            for j = 1:num_vacant_plots
                if resident_vars_reshaped(i,j) > 0.5
                    total_cost = total_cost + cost_map(i,j);
                end
            end
        end
        total_cost_values(m_idx) = total_cost;

        % 计算腾退院落数量

```

```

        courtyard_vars
x(num_residents_vars+1:num_residents_vars+num_courtyard_vars);
        evacuated_courtyards(m_idx) = sum(courtyard_vars > 0.5);

        % 计算相邻院落对数量
        adjacent_vars = x(num_residents_vars+num_courtyard_vars+1:end);
        adjacent_pairs(m_idx) = sum(adjacent_vars > 0.5);

        % 计算总租金收益
        courtyard_rent = 0;
        for i = 1:num_courtyards
            if courtyard_vars(i) > 0.5
                courtyard_rent = courtyard_rent + courtyard_annual_rent(i);
            end
        end

        % 相邻院落加成
        adjacent_rent = 0;
        adjacent_idx = 1;
        for i = 1:num_courtyards
            for j = i+1:num_courtyards
                if adjacency_matrix(i,j) == 1
                    if adjacent_vars(adjacent_idx) > 0.5
                        adjacent_rent = adjacent_rent + 0.2 *
(courtyard_annual_rent(i) + courtyard_annual_rent(j));
                    end
                    adjacent_idx = adjacent_idx + 1;
                end
            end
        end

        total_rent = (courtyard_rent + adjacent_rent) * 10; % 10 年
        total_rent_values(m_idx) = total_rent;
    else
        % 无可行解
        objective_values(m_idx) = NaN;
        total_cost_values(m_idx) = NaN;
        total_rent_values(m_idx) = NaN;
        evacuated_courtyards(m_idx) = NaN;
        adjacent_pairs(m_idx) = NaN;
    end
end
end

```

附录 4：问题四算法程序

程序编号	4	程序说明	智能搬迁决策程序	编程软件	python
<pre> import gurobipy as gp # 使用 Gurobi 的 Python 接口 from yalmip import * # 需确认 Yalmip 在 Python 中的兼容性,或替换为 PuLP/Pyomo class RelocationModel: def __init__(self): self.G_s = None # 满意度阈值 self.G0 = None # 基准满意度 self.m = None # 性价比参数 self.m_b = None # 性价比拐点 self.vacant_plots = [] # 空置地块集合 self.role = None # 用户角色 self.consider_cost_performance = False # 是否考虑性价比 def initialize_compensation_model(self): """初始化搬迁补偿模型""" self.G0 = 80 # 示例值, 需根据实际数据设定 self.G_s = 85 self.m_b = 1.2 # 示例性价比拐点 def filter_vacant_plots(self): """筛选满意度$\geq G_0$的空置地块""" self.vacant_plots = [plot for plot in self.vacant_plots if plot['satisfaction'] >= self.G0] print("筛选后的空置地块: ", self.vacant_plots) def solve_problem2_with_gurobi(self): """调用优化求解器求解问题二模型""" try: model = gp.Model("Relocation_Problem2") # 添加变量和约束 (需根据实际模型补充) x = model.addVar(vtype=gp.GRB.BINARY, name="x") model.setObjective(x, gp.GRB.MAXIMIZE) model.optimize() print("求解结果: ", model.objVal) return True except Exception as e: print("求解失败: ", str(e)) return False def calculate_max_m(self): """反推 $m < m_b$ 时的最大拐点性价比""" self.m = self.m_b * 0.9 # 示例计算, 需替换实际算法 </pre>					

```

print(f'调整后的性价比参数 m*: {self.m}')

def user_interaction(self):
    """用户交互流程"""
    # 初始化
    self.role = input("请输入用户角色（政府/开发商）： ")
    self.consider_cost_performance = input("开发商是否考虑性价比？ (y/n): ").lower() == 'y'

    if self.role == "开发商" and self.consider_cost_performance:
        self.initialize_compensation_model()
        self.filter_vacant_plots()

    # 求解问题二模型
    if self.solve_problem2_with_gurobi():
        has_angle = input("是否存在算角？ (y/n): ").lower() == 'y'
        if has_angle:
            self.calculate_max_m()
            change_m = input("是否变更性价比参数？ (y/n): ").lower() == 'y'

            if change_m:
                self.m = float(input("请输入新的性价比参数 m: "))
                print("参数已更新，重新计算...")
                # 重新求解模型
                self.solve_problem2_with_gurobi()
            else:
                if self.m > self.m_b:
                    print("输出跨空流搬迁方案")
                else:
                    print("未生成满足条件的方案，请重新输入参数")
        else:
            print("求解失败，请检查模型参数")
    else:
        print("执行非性价比敏感流程...")

if __name__ == "__main__":
    model = RelocationModel()
    model.vacant_plots = [
        {'id': 1, 'satisfaction': 85},
        {'id': 2, 'satisfaction': 75}
    ] # 示例数据
    model.user_interaction()

```